

ra8ci

One control plane for CI, the lab, and the board

Approved architecture and implementation contract | 23 September 2026

The proposal in one sentence. A single Go product provides a command-line interface, a server in a protected Proxmox VM, and outbound-connecting agents. It controls disposable GitHub runner capacity, owns durable task and board lifecycles, records fine-grained measurements in colocated PostgreSQL, invokes Terraform and Ansible for provisioning, and runs named Zig build and quality tasks behind familiar just recipes.

Status: Brighton approved the architecture and Go choice. The pushed feature branch `ci/ra8ci-implementation` is rebased on `origin/dev` at `71feca26c`; the last code snapshot before this documentation refresh is `125f026c`. The implementation reference records the verified current state, the unverified Proxmox Linux/Windows baseline, the Windows secret-propagation blocker, and the concrete integration handoff. Production ra8ci dispatch and deployment are not complete. The active Zig build-graph work remains out of scope.

Revision, 23 September 2026, evening. The demand signal is re-specified after review. `workflow_job` webhooks plus just-in-time runner registration are now the primary path; the Actions Runner Scale Set Client is retained only as a feature-gated second adapter behind one internal interface. An unclaimed-runner reaper becomes a required component. The Windows lab credential moves out of the environment. Self-hosting a forge is rejected as an answer to this problem. See “Demand signal: webhooks and just-in-time runners”, “Second migration: ra8ci posts its own check runs”, the migration spine, and the decision register.

Audience: Brighton, the planner AI, implementers, lab operators, and future CI maintainers.

Contents

Executive brief	3
Goals, boundaries, and current system	4
User journeys	5
Product topology and deployment	5
Task contract and Zig integration	8
Durable execution and scheduling	9
Board commons: human, CI, and AI coordination	10
Database and audit design	11
API, CLI, and operations	13
Alternatives and trade-offs	13
Migration with proof at every boundary	14
Implementation appendix	16
Executable implementation contract	19
Implementation reference	27
Decision register and discussion prompts	47
Evidence and sources	48

Executive brief

What changes for a person using the repository

The familiar command remains a just recipe. A developer can use just format, just test, or just hil. Those recipes call the corresponding ra8ci task name. The same task can be called directly by a person, a git hook, a CI trigger, or an AI agent. There is one definition of what the task does, what it needs, how long it may run, and which evidence it must produce.

Simple, safe local tasks can run on the machine that invoked them. For GitHub Actions, GitHub remains the workflow scheduler: ra8ci receives demand for runner capacity, creates eligible disposable guests, and tracks the tasks those jobs invoke. Hardware work enters the ra8ci board queue. A person can see not only that a run is slow but whether it spent its time waiting for a runner, creating a VM, downloading dependencies, compiling, waiting for the board, or executing a particular step.

The board remains one-at-a-time. A human can request it with a reason and duration. An active CI or AI task receives a yield request and stops at its next declared safe checkpoint. The lease is not forcibly stolen during a flash, reset, or measurement. The person sees the holder, remaining time, and queue. Every transition is audited.

Chosen technology and deployment boundaries

- **Implementation:** Go for one source tree and one binary name with server, agent, and CLI modes. Build separate static binaries for Linux amd64 and Windows amd64; do not claim that one executable file runs on both operating systems.
- **Persistent state:** PostgreSQL, written by the server. Agents never receive database credentials. The schema has typed lifecycle tables and an append-only audit stream.
- **GitHub integration:** ra8ci uses GitHub's Go Actions Runner Scale Set Client as the controller for an ephemeral runner scale set. Each disposable Linux or Windows guest runs the official GitHub Actions runner for one job plus ra8ci agent for our task, telemetry, and board contracts. ra8ci does not reimplement the GitHub job worker.
- **Transport:** Authenticated HTTPS between CLI, server, and outbound-connecting agents. Use server-sent events or chunked HTTP for live logs at first; a versioned bidirectional stream can be added only if measured needs justify it.
- **Provisioning:** ra8ci owns the run state and authorizes every create or destroy. Terraform remains the declarative guest mechanism; Ansible remains guest and fleet provisioning. The direct qm lifecycle is retired when parity is proven.
- **Build path:** Semantic ra8ci task names invoke the current build implementation. As epic 857 completes, those implementations move to zig build steps. just remains the convenient wrapper; ra8ci does not implement another build graph.
- **Deployment:** One protected, persistent control VM on Proxmox (VM ID in the 9000+ range) runs ra8ci server and PostgreSQL together; use a Unix socket for their local database connection. Agents run in disposable Linux and Windows guests and at the board boundary. Repository jobs never run on the control VM or Proxmox host. No ra8ci agent listens for inbound jobs.

Why this shape

The hardest failure today is split ownership: one disposable lab path runs Terraform then Ansible, while another server-side path directly clones guests with qm. A durable controller gives each run one identity and one recovery path. GitHub's scale-set client lets that controller supply native GitHub runner capacity without copying the runner's job protocol. The controller also provides the timestamps needed to see where CI time actually goes. Keeping Zig responsible for build dependencies avoids duplicating its graph. Keeping Terraform and Ansible responsible for infrastructure convergence retains the established safety boundaries.

What is not promised yet

The first version should not claim automatic optimal scheduling, arbitrary interruption of HIL tests, a web dashboard, or complete removal of every helper script. Nor does the scale-set client make ra8ci itself a replacement for the official Actions worker. Those are different responsibilities. This document proposes a simple API and CLI first; a visual console is optional.

Goals, boundaries, and current system

Goals

1. A named task behaves the same whether invoked through just, the CLI, a hook, or CI.
2. Every execution has an owner, deadline, host, result, log, steps, and resource samples.
3. The server can reconcile an interrupted run after its own restart without inventing success or silently destroying a nonmatching guest.
4. One board lease is active at a time; humans, CI, and AI agents can cooperate through a visible queue and safe yields.
5. Analytics identify the critical path and the resource profile of slow steps before anyone changes scheduling tiers.
6. The CMake-to-Zig transition can change implementation commands without changing task identities or erasing historical comparison.

Non-goals

- Implementing the Zig build graph, porting firmware libraries, or changing epic 857
- Replacing Terraform state, Ansible roles, OpenBao, or the persistent runner fleet declaration before a proven cutover.
- Reimplementing the official GitHub Actions worker or executing arbitrary `uses`: actions inside the control VM.
- Giving an agent general-purpose shell access to the Proxmox host.
- Making a database outage look like a successful HIL run.
- Removing a script before its behavior and callers move in the same MR.
- Defining issues, epics, or stacked MR topology; the separate planner owns those.

Observed repository baseline

The published dev branch inspected for this draft was a7666d637. Its disposable lab driver uses run-local Terraform state, template checks, an isolated bridge, Ansible playbooks, and strict cleanup identity checks. A separate server-side runner uses direct `qm` commands and host networking changes. The Linux playbook stages a source archive and runs just ci in the pinned container. The Windows playbook prepares a disposable Windows guest and runs just ci there. The persistent fleet is declared in `infra/fleet.yml`: ARC on k3s, Docker runners on NAS and Windows/WSL, and a dedicated HIL listener on dev. Workflows select these with `runs-on` labels. Ansible installs the official runner, whose listener starts a worker per job. The dedicated HIL listener and bench are separate from ordinary scalable runners.

The accepted Phase 0 inventory on ci/orchestrator recorded 588 helper rows: 557 used, 31 dead, and zero unclear. That is a historical baseline, not a current-tree count. Since then the lane has removed dead helpers and absorbed five checkers; at implementation commit d928cb607, the current checkout has 532 tracked `.sh/.py` files and 28 `just/*.just` files. The machine-readable inventory and work-unit ledger must be reconciled with that current tree before the remaining planner units are cut. These raw counts do not mean every file becomes a public task: Terraform/Ansible stay as the provisioning layer, deliverable programs remain out of scope, and some helpers may be retained with an explicit reason.

User journeys

Fast local check

A developer runs just format. The recipe invokes ra8ci format. The CLI resolves the repository root and requires the embedded task-catalog digest to match the checkout's reviewed manifest. The first release uses pinned binary and catalog digests, not an in-app release-signing protocol or compatibility adapters. A mismatch fails visibly rather than silently using the wrong task. The CLI creates a local run identity, executes bounded steps, prints logs directly, and records the child's exit code; its own stable exit category is documented in the implementation contract. Safe local tasks such as format and lint run even when the server or PostgreSQL is unavailable. Their results are durably spooled on the invoking host as explicitly unsynced receipts, then uploaded idempotently when service returns. A synced receipt remains labeled local/offline and is never promoted into a CI gate attestation. Board and dispatched tasks fail closed.

Disposable CI run

A GitHub workflow selects an ra8ci-managed runner scale set with `runs-on`. GitHub owns the workflow queue and offers job demand to the ra8ci controller. The server records its capacity decision before provisioning, creates a disposable guest through the restricted Terraform/Ansible path, and passes a short-lived just-in-time runner configuration to the official runner in that guest. The runner accepts one GitHub job and reports its native logs and status to GitHub; ra8ci agent records our named task steps, resources, and board use. The guest is then drained and destroyed. A failed cleanup remains a visible incident requiring reconciliation; it is never reported as clean success.

Human asks for the board

A human runs ra8ci board take -why "debugging power sequencing" -duration 30m. The server records a waiter with human priority. If CI holds the board, the server sends a yield request. The holder reaches a safe checkpoint, records its completed phase, puts the board in a neutral state, releases the lease, and requeues its continuation. The human sees the wait position and then the granted lease. If the holder never reaches a checkpoint, it keeps the board until its deadline or safety recovery; priority is not permission to interrupt electrical work.

AI agent works beside a human

The AI can run emulator and analysis tasks without a board. Its HIL task waits behind the human. It cannot bypass the server or proceed because a local lock file appears stale. When the human releases, the queued task is revalidated before acquiring a lease, because the checkout, image, or board state may have changed while it waited.

Product topology and deployment

One product, three modes

- ra8ci server: persistent HTTP API, scheduler, durable state machine, board authority, audit writer, deadline enforcer, and report query service.
- ra8ci agent: outbound authenticated connection, capability/host-fact report, local bounded process supervisor, log/resource sampler, step result producer, and safe-yield client.
- ra8ci task: human/automation CLI that executes approved local tasks or submits a dispatched run. Board and report subcommands are clients of server authority.

A single Go module and shared task schema avoid inconsistent behavior. The control VM starts as one server process and one colocated PostgreSQL service; separate server workers are a future split only if measured load or isolation requirements demand them. There is no need to split the ra8ci binary

merely because modes run on different hosts. The official GitHub runner remains a separate upstream program inside each disposable job guest.

GitHub Actions control and execution

GitHub Actions has two distinct planes. GitHub evaluates workflow YAML, queues jobs, matches runs-on labels, and reports workflow results. A self-hosted runner registers with GitHub, listens for a job, starts a worker, executes shell and uses: steps, and reports native logs and status. A process that merely posts a check run or exposes an HTTP API is not a GitHub Actions runner.

ra8ci learns that a job wants capacity from a workflow_job webhook delivered to a GitHub App endpoint on the control VM, and mints the runner credential with the documented generate-jitconfig REST endpoint. ra8ci maps demand to a capacity reservation, provisions a guest through the existing Terraform/Ansible layer, enrolls ra8ci agent, and starts an official ephemeral runner in that guest with the just-in-time configuration. The runner accepts at most one job. GitHub continues to schedule its workflow steps; ra8ci schedules capacity and its own named tasks. These are separate queues with correlated IDs, not competing owners of one job.

The controller must account for capacity requested, provisioning, registered/idle, busy, draining, deregistered, and destroyed. It never advertises capacity as usable before the guest is healthy, and never treats an assignment, registration, or delete request as proof that a job ran or cleanup completed. Reconcile GitHub runner state, Proxmox guest identity, Terraform state, and ra8ci agent receipts after any lost acknowledgment. A stopped runner cannot be killed just to scale down while its worker is busy. Linux and Windows guests require separate images and capability labels; HIL stays in its dedicated lane until the board lease migration is proven.

No direct GitHub runner wire-protocol implementation is authorized by this design. The next section specifies where demand comes from and what must exist before a credential is minted.

Demand signal: webhooks and just-in-time runners

Decision. The primary demand source is the workflow_job webhook; the one-job runner credential comes from POST /repos/{owner}/{repo}/actions/runners/generate-jitconfig. Both are generally available surfaces that GitHub documents as the basis for custom autoscaling. The Actions Runner Scale Set Client remains in the tree as an optional second adapter behind the same internal interface, feature-gated and off by default. It is promoted only if it leaves public preview with a stated compatibility policy and a maintained integration test rig. The original design made one public-preview Go package the single point of failure for all dispatch; that is the defect this revision removes.

Both adapters normalize into one internal demand event. No adapter-specific identifier may become a primary key in PostgreSQL, and no scale-set protocol state may leak into the run state machine. Failover is an operational action, not a code change: disable one intake, reconcile outstanding reservations, enable the other.

```
type DemandEvent struct {
    Source          DemandSourceKind // webhook or scaleset
    DeliveryID      string           // GitHub delivery, for idempotency
    WorkflowJobID   int64
    RunID           int64
    Attempt         int
    Repository      string
    HeadSHA         string
    Labels          []string
    ObservedAt      time.Time
}

type DemandSource interface {
    Observe(ctx context.Context, ev DemandEvent) error
    Cancel(ctx context.Context, workflowJobID int64) error
    Reconcile(ctx context.Context) error
}
```

```

}

type RunnerDelivery interface {
    MintOneJobCredential(ctx context.Context, spec RunnerSpec) (JITConfig, error)
    Revoke(ctx context.Context, runnerID int64) error
}

```

Delivery is a hint, not a queue. Treat `workflow_job.queued` as at-least-once. Deduplicate on workflow job ID plus attempt, persist the GitHub delivery ID, and run a reconciliation pass that lists queued and in-progress Actions jobs for open pull-request head SHAs and opens the demand rows no webhook delivered. A lost delivery must degrade to a late run, never a silent one. Handle `completed` and `cancelled` as cancellation inputs, but never as the only ones; the database stays authoritative.

The unclaimed-runner reaper. A just-in-time runner is single-use only after it has received a job. A guest provisioned for a job that is cancelled, relabelled, superseded by a new push, or delivered twice will otherwise sit idle and registered. Every reservation therefore carries an explicit unclaimed deadline. On expiry `ra8ci` revokes the runner registration, destroys the guest, releases any bench lease, and marks the attempt abandoned rather than leaving capacity available for unrelated work. This reaper is a required component of the first dispatch release, not a later optimization: it is the one thing the scale-set listener provided for free.

Ordering. Mint the just-in-time configuration as late as possible: after capacity admission, after bench admission, after the guest's network policy is in place, and after its identity channel exists. Runner labels stay narrow, naming OS, image generation, architecture, trust tier, and bench class. No generic self-hosted pool ever serves untrusted pull-request content.

Prototype gates, unchanged in substance. Before dispatch depends on either adapter, an isolated prototype must verify repository-scoped authorization, Windows guest registration, label matching, one-job teardown, cancellation, quota behavior, and recovery of pending jobs from failed provisioning. Under the webhook path it must additionally verify signature validation, duplicate and out-of-order delivery, and reaper expiry.

Trust boundaries

The control VM has authority to request lab lifecycle actions, not carte-blanche root access to the Proxmox host. A dedicated, least-privilege Proxmox API credential or narrowly scoped provisioner invokes approved Terraform and Ansible workflows. Template, bridge, pool, datastore, run marker, and stopped-state checks remain enforced at the provisioning boundary. Credential delivery through the existing secret system must be verified before deployment; neither an API credential nor OpenBao access is assumed to exist on dev. No Proxmox credential is copied to a guest.

GitHub App credentials belong only to the control VM and are scoped to the repository or organization and permissions actually required by the scale set. Short-lived registration and just-in-time configuration material is treated as a secret and passed only to the intended guest. GitHub job tokens and repository secrets remain confined to the disposable worker. Workflow YAML and checked-out repository code are untrusted inputs: neither may select a Proxmox API operation, inject provisioner arguments, reach the control VM's database socket, or execute on the Proxmox host. Named task arguments are validated against a reviewed schema; no shell-concatenated command reaches a privileged boundary. Audit authorization failures, token issuance, VM lifecycle actions, and runner cleanup without logging secret values.

The Windows lab credential is not carried in the environment. `RA8_LAB_WINDOWS_PASSWORD` leaves the runner script's inherited environment; the value lives in a root-owned file with restrictive permissions on the Proxmox runner host and is read at the point of use by the process that needs it. It is never sourced into a shell, never exported, and never passed through `sudo`. Verification reports presence as a boolean and nothing else. No log line, process listing, crash dump, or guest copy may carry the value.

Agents receive task-scoped credentials and a capability allowlist. A disposable guest agent first enrolls against a capacity reservation and verified VM identity with health-only authority; it has no run ID, source SHA, task, or board permission until the server correlates a GitHub job and issues a separate job grant. A Windows guest never receives a token that can create Proxmox VMs. The board-side agent cannot mint its own lease. Unlike disposable guests, the persistent board agent has an operator-enrolled host identity with rotation and revocation; it is not bound to a run ID.

PostgreSQL lives on persistent storage in the protected control VM, not inside a disposable CI guest. ra8ci connects through a local Unix socket; no database port is exposed to runners. The app role can insert audit records but not update or delete them. This is one failure domain, so off-VM encrypted backups, retention, schema migration, restore drills, and a named backup destination are deployment requirements, not optional analytics polish. A database outage stops dispatch and board grants but does not stop safe local tasks.

Host and resource model

Each agent reports OS/architecture, host class, CPU cores, RAM, available disk, current load, supported tools, board reachability, and an agent version. The server additionally stores the declared budget from `infra/fleet.yml`. Scheduling is bounded by the declared budget; instantaneous host facts refine placement but never authorize consuming more than the declaration. Runner capacity and quiet-hours drains remain the fleet layer until explicitly migrated.

Task contract and Zig integration

Task definition

A task has a stable semantic name, schema version, tier (required, optional, nightly), execution scope (local, runner, Linux VM, Windows VM, HIL), supported OS/architecture, required capabilities, deadline, board requirement, safe checkpoints, step list, output contracts, retry policy, and resource hints. Command arguments are structured, not shell-concatenated strings. The catalog is embedded in the binary; the repository carries a reviewed manifest of its digest and compatible schema range. A run records the binary, catalog, manifest, and source digests. A mismatch refuses execution unless an explicitly approved compatibility adapter exists.

A task name is not a source file name. For example, `format`, `unit-test`, `cross-build`, `lint`, `emulator`, `HIL`, and `report` are stable concepts. The current command adapter may invoke an existing script only during the executor stage. Each later migration replaces that adapter with a real implementation and deletes the old script and repoints the just recipe in the same MR.

Zig owns build dependencies

ra8ci invokes selected zig build steps with a pinned Zig toolchain, explicit target/configuration, and an out-of-tree cache/prefix. It does not parse `build.zig` to recreate the dependency graph. Zig is responsible for parallel build steps and caching within a build. ra8ci is responsible for cross-task placement, deadlines, board access, and longitudinal evidence.

The task identity and schema survive the CMake retirement. Historical records include `engine=cmake` or `engine=zig`, Zig/CMake version, target, configuration, and build-graph digest. Reports compare like with like; a change of engine is a segmentation boundary rather than an unexplained speedup. The 857 parity gates remain authoritative until the Zig route preserves analysis databases, coverage, stack-usage, and other required artifacts.

just remains the front door

Just recipes call ra8ci by task name and pass explicit arguments. A recipe does not duplicate task policy or decide which VM to use. Hooks and workflows use the same names. A direct ra8ci invocation is equivalent; it is not required to have just installed on an agent if the task does not itself need just.

Durable execution and scheduling

Run state machine

A run moves through accepted, queued, provisioning, ready, running, draining, cleaning, and a terminal state: succeeded, failed, timed_out, cancelled, or cleanup_failed. Each transition is transactional and emits an audit event. A task attempt is distinct from the logical task so a retry does not overwrite the original timing or result. There is no “unknown equals success” state.

Provisioning actions have operation IDs. Before executing an external action, the server commits intent and expected identity. Terraform state is run-scoped but stored in a persistent, access-controlled directory keyed by run ID, with encrypted backup and an operation-to-state-path mapping in PostgreSQL; it is never a throwaway process temp directory. After an interruption the server reconciles against that state, Proxmox markers, and agent facts, including the case where a provisioner died before publishing state. Replaying a create/destroy call without checking identity is forbidden. Acknowledgment loss may yield an unknown result; reconciliation, not blind retry, resolves it.

Queue and placement

Initially use explicit rules, not a learned scheduler. For ra8ci-owned tasks, enqueue by tier and priority, with age-based fairness and per-host/per-profile concurrency limits. For GitHub workflows, GitHub retains job ordering and matching; ra8ci reports available scale-set capacity and chooses which eligible guests to provision. It must not promise to reprioritize a job already assigned by GitHub. Choose guests/agents with matching OS, capability, declared budget, free memory, and quiet-hours state. Preserve a lane for quick required checks so large nightly work cannot monopolize every worker. Board-dependent tasks acquire a board lease only immediately before their first board step; emulation, compilation, and packaging happen before the lease.

Top-level run intervals form a disjoint wall-time partition: queued, provisioning, setup, executing, and cleanup. Board wait and board hold are labeled subintervals inside executing, not extra time added to it. Overlapping work on different agents is shown as a timeline and dependency graph, not summed into run wall time. This is essential: CI taking an hour is not a useful optimization target until the hour is attributed.

Deadlines, cancellation, and logs

The server holds an absolute ra8ci task deadline and the agent enforces a local monotonic timeout. GitHub may also cancel its workflow job; the GitHub runner owns that outer cancellation, and ra8ci must correlate it to a task cancellation receipt rather than report a second, contradictory success. For non-board tasks, timeout or cancellation first requests process-tree termination, waits a bounded grace period, and then kills the remaining tree; board tasks instead obey the phase-aware cancellation and recovery rules below. This behavior needs separate Linux and Windows tests. A skipped test is never a pass; it is a distinct result with an explicit reason and policy decision.

Logs are streamed in numbered chunks with run/task/attempt/step IDs. A reconnect resumes from the last acknowledged sequence. The server stores an indexed excerpt and durable artifact pointer; giant stdout blobs do not live in PostgreSQL rows. Every result records truncation and artifact integrity. The task row is terminal only after exit code and log finalization are durably recorded, or after a clear lost-agent reconciliation outcome.

Board commons: human, CI, and AI coordination

Lease invariants

1. At most one active board lease exists. The database enforces this invariant transactionally; a local lock is not authority. The grant is unusable until the board-side agent durably acknowledges its generation.
2. Priority is human above CI above AI agent, but active work is cooperatively yielded, never yanked in the middle of an unsafe phase.
3. A waiter does not touch hardware. An emulator task runs first and does not hold the board.
4. Every lease has a requested duration, absolute expiry, heartbeat, holder identity, reason, and generation token.
5. Expiry ends the holder's authority and moves the board into recovery-required state. No new lease generation is granted until the board-side agent confirms a neutral state; an offline or unresponsive agent keeps the board unavailable.
6. A stale holder cannot act after its generation is superseded. The board-side agent durably records its highest seen generation, validates the current lease token before each board-touching step, and fails closed when it cannot validate it; an already-started bounded safe segment completes before recovery. A database restore behind the agent's high-water mark forces quarantine.
7. Take, grant, extend, yield request, checkpoint, release, expiry, recovery, and denied action are audited. All software-mediated flash, reset, probe, UART, relay, and power operations cross the board agent's serialized local gate. Direct physical, root, or locally attached-probe access cannot be cryptographically fenced and needs an operator physical-control procedure.

Queue and handoff

A human request joins ahead of CI and AI waiters. A yield request is visible to the holder and the human. Each task declares checkpoint boundaries and a maximum noninterruptible segment. At a checkpoint it stops board I/O, flushes results, restores the safe board state, releases, and requeues a continuation behind the human. A task that cannot safely resume declares itself nonresumable; yielding then ends that attempt as interrupted and a later attempt starts from a clean image. No generic pause/resume illusion is presented.

Fairness needs more than strict priority. While a human is waiting, new lower-priority work cannot jump ahead. Within one priority class use FIFO with bounded aging. A human who keeps extending indefinitely is visible, and extensions require a reason and policy limit. A second human cannot silently preempt the first.

Dynamic yield budget and cancellation

Three clocks remain distinct: a HIL test's validity deadline, the time from a human yield request to a safe checkpoint, and the safety limit of an indivisible flash or recovery operation. The current per-app hil.conf files declare observation/probe windows, sometimes longer than 30 seconds; they do not bound total setup and flash or authorize interruption. ra8ci imports those values as test deadlines and fallback evidence only where the declared step matches. A board task must also declare a maximum safe-step and restore/probe duration. If comparable request-to-neutral history has sufficient samples, a conservative high quantile plus margin supplies a visible dynamic handoff estimate; otherwise the declared bounds are used. If a board task has no such bounds, its ETA is unknown and automatic dispatch is rejected. Thirty seconds remains a handoff target and a default process-stop grace only for a separately declared cancel-safe experiment; it is never a substitute safety bound. The estimator never shortens a declared safety bound, and its cohort, sample count, and age are shown to the requester.

Thirty seconds is the default handoff target, not a guarantee that permits a half-written flash. Once a human waits, no new lower-priority board step starts. If an active flash, erase, verify, or required recovery exceeds the target, the holder finishes that indivisible phase, reports the overrun and revised

ETA, then yields. If the phase hangs or board state cannot be proved neutral, the board enters recovery-required or quarantined state rather than being handed to the human. The same rule applies to task validity expiry: invalid result does not imply safe board release.

For checkpointed HIL, the interrupted attempt ends with a distinct preempted result and requeues for a fresh run from a verified baseline, unless that specific task declares a tested resumable checkpoint. For an open-ended AI experiment, a yield first requests graceful stop, then terminates its process tree after a bounded drain if its current phase is cancel-safe. Stopping the host program is not enough: the board-side agent fences its access, resets or reflashes the board according to a declared restore policy, probes the result, and only then releases it. An open-ended firmware image cannot be assumed to recreate RAM, peripheral, or interaction state when reflashed; its owner must restart the experiment. Human-held work is notified of later requests but is never force-preempted by CI or AI.

Board session metadata records the immutable firmware artifact digest, flash arguments, board/fixture revision, owner, current phase, last safe checkpoint, restore image/action, and retry class. Queryable identity and state belong in typed columns; JSONB holds versioned, variable metadata, not firmware bytes. Artifacts live in durable content-addressed storage. A failed restore quarantines the board and requires operator recovery.

Failure and recovery

On holder crash, the server waits for expiry and asks the board-side agent to perform a reviewed recovery sequence. On server or PostgreSQL outage, the board-side agent stops at the next safe point and does not grant a new lease locally. If a hardware operation is already in progress, it completes only the bounded safe segment. After recovery, the server reconciles board state before granting a new token. These rules deliberately favor safety over throughput.

The first board version should preserve `bench.sh` hold, free, extend, status, and journal behavior. Cutover keeps the old flock as the sole authority while the new lease path runs in shadow, then drains old holders, disables direct access, and enables the new agent gate; the two authorities are never independently active. Rollback requires all new leases drained. Acceptance testing includes two concurrent claimants, human yield during flash, lost agent, server restart, database outage, expiry, stale token, direct-bypass attempt, and safe release failure.

Database and audit design

Core relational tables

- `runs`: run ID, trigger, actor, repository, branch, commit, source snapshot digest, task catalog digest, creation/start/end, status, and parent run.
- `tasks`: logical task ID, run ID, stable task name, tier, host class, placement, enqueue/start/end, deadline, terminal result, and board requirement.
- `task_attempts`: attempt ID, task ID, agent, start/end, exit code, timeout/cancel/lost-agent reason, host facts at start, and implementation engine.
- `task_steps`: attempt ID, stable step key, ordinal, start/end, exit/result, exclusive phase label, and artifact links.
- `task_edges`: run ID, predecessor task/step, successor task/step, and dependency reason; these edges form the recorded execution DAG needed for critical-path analysis.
- `resource_samples`: attempt/step, monotonic offset, CPU time, process CPU utilization, process-tree RSS and peak RSS, host load, RAM available, disk and network I/O. Keep a documented sampling interval and record gaps.
- `board_waiters`: request ID, class, actor, reason, enqueue time, requested duration, state, and queue order.
- `board_leases`: lease ID, board ID, holder, generation token, grant/expiry/release times, reason, requested duration, yield state, and end reason.

- `board_sessions`: session ID, owner, board/fixture revision, lease generation, phase, cancellation and retry class, restore policy, current and baseline image digests, last safe checkpoint, and versioned JSONB metadata.
- `board_yield_samples`: session/step ID, request/checkpoint times, observed latency, task/catalog/image/board cohort, safety overrun, and exclusion reason; failed or censored requests are retained but not passed off as completed latency samples.
- `agents`: identity, host class, enrollment, capabilities, version, last heartbeat, and declared capacity.
- `github_jobs`: GitHub installation/repository, workflow run and attempt IDs, job ID, requested scale set and labels, runner identity, immutable commit, GitHub conclusion, and correlation to the ra8ci run. Store references and status, not GitHub tokens or unredacted payloads.
- `runner_lifecycles`: capacity request, guest and Terraform identity, JIT issuance, registration, first job, last worker state, deregistration, destruction, and any reconciliation/cleanup failure.
- `artifacts` and `log_chunks`: integrity digests, content location, byte size, sequence, retention, and redaction status.
- `audit`: immutable event ID, wall time, actor, action, target, correlation/run ID, request ID, outcome, previous/new state, and structured reason.

The board uniqueness constraint and state transitions must be tested against concurrent transactions. PostgreSQL row locks or queue-safe selection may coordinate waiters, but the safety invariant lives in a constraint and transactional state change, not in timing luck. The app's normal database role cannot mutate audit history.

What the first reports answer

- ra8ci report slow ranks median, p90, and total wall time for comparable tasks over a chosen window, with sample count and failure rate.
- ra8ci report critical-path computes the longest dependency path from recorded `task_edges` and step durations, with missing edges or clock uncertainty called out rather than guessing from the longest individual task.
- ra8ci report idle-time separates queue wait, provisioning wait, board wait, and host idle capacity.
- ra8ci report runners separates GitHub queue delay, scale-set demand-to-provision, guest boot/setup, runner idle, job execution, and cleanup. GitHub-owned timestamps and ra8ci-observed timestamps carry provenance; missing GitHub events are shown as gaps, not invented durations.
- ra8ci report resource-fit identifies long low-CPU steps, but also reports RSS and I/O so parallelization does not create memory or disk contention.
- ra8ci report board shows utilization, wait by priority class, yield latency, expired leases, and time spent in recovery.
- ra8ci report regress compares equivalent commit ranges, profile, engine, task schema, and cache state.

No task changes from required to optional on a hunch. A tier change is reviewed with sample size, failure consequence, dependency impact, and a recorded rationale. Analytics is evidence for a decision, not an automatic gate weakening mechanism.

Retention and privacy

Audit and run metadata persist longer than raw logs and high-frequency samples. Define retention by class and estimate storage before deployment. Secret values, OpenBao tokens, command environment, and private board data are redacted before persistence. The audit stream records that a secret was accessed or a provisioner was invoked, never the secret value. Restore drills must prove both database and artifact references survive.

API, CLI, and operations

Initial CLI surface

- ra8ci format, lint, test, build, emulator, and HIL task names; `-local` or `-dispatch` when both modes are meaningful.
- ra8ci run status, logs, cancel, retry, and explain for run visibility.
- ra8ci board take, release, extend, status, and queue.
- ra8ci report slow, critical-path, resource-fit, and board.
- ra8ci agent and ra8ci server for deployment modes.

Human output is concise by default; `-json` gives stable machine-readable schemas. Exit codes distinguish task failure, unavailable server, denied policy, timeout, cancelled, and incomplete evidence. The API and agent protocol are versioned. A new server must tolerate an older agent for a bounded transition window, or explicitly drain it before upgrading.

API and access control

Use authenticated HTTPS, short-lived agent identity, and role-based permissions: observer, task submitter, board human, operator, and provisioner. Humans may view their authorized logs and request board time; agents cannot request human priority; only operators can approve exceptional keep-guest or cleanup recovery. Every authorization failure is audited without leaking secrets. The server binds to a protected management interface and is not exposed directly to the lab guest subnet.

Operational readiness

A health endpoint distinguishes “process alive” from “database writable, scheduler able to commit, and audit durable.” Startup performs schema compatibility checks and refuses unsafe downgrade. Backups, disk pressure, expired certificates, stalled agents, stuck cleanup, lease recovery, and logging lag have explicit alerts. An operator can put the server in drain mode, stop new dispatch, and let current safe segments finish.

GitHub integration health separately reports scale-set session health, authentication expiry, runner registration failures, assigned-versus-available capacity, and orphaned guests. A GitHub outage does not revoke a human’s existing safe local task, but no new GitHub job is presumed assigned; board authorization still requires ra8ci’s own healthy server and database. A control VM outage leaves GitHub jobs queued or pending according to GitHub’s own limits, not silently passed. Recovery replays durable capacity decisions and guest identities before starting new provisioners.

Alternatives and trade-offs

Go, Rust, or Zig for ra8ci

Decision: Go. It provides a straightforward portable server, process supervision, HTTP service, SQL client, and cross-compiled Linux/Windows binaries from one codebase. The Zig migration does not require the orchestrator itself to be Zig: Zig remains the build system and firmware/library language, while ra8ci coordinates tasks and records evidence. Rust can offer tighter memory control but raises implementation complexity for a rapidly changing scheduler. Zig would unify languages but couples the control plane to the actively changing compiler/build stack. A language change now requires a new architecture decision and migration plan.

PostgreSQL versus a local file database

Decision: PostgreSQL. Durable concurrent state transitions, queue claims, relational analytics, and access controls fit the server/agent architecture. Centralize writes through the server; “many agents” is a reason for durable shared state, not for distributing database credentials. A local database is

simpler for one process but complicates failover and multi-host audit. The cost is a persistent service and backup obligation.

One server versus microservices

Decision: one server process first. It gives one lifecycle authority and fewer failure seams. Worker-like components can be internal packages with narrow interfaces. Split processes only when measured reliability or resource isolation requires it. Independent microservices from day one would multiply protocol, deployment, and reconciliation burdens before the workload is understood.

Direct Proxmox API versus Terraform/Ansible

Decision: Terraform/Ansible remain the production mechanisms. ra8ci owns when, why, and which run, and verifies results. The current direct qm runner is an alternate provisioning path and should retire only after a verified replacement exists. Keeping the existing guardrails avoids two sources of truth for VM and network state. The Go Proxmox client may reconcile exact identity but is not authorized as the production mutation backend. Direct API mutation requires a separate safety review and architecture decision, not an incidental optimization.

GitHub runner replacement versus scale-set control

Decision: scale-set control with the official runner in disposable guests. Replacing the runner binary itself would require ra8ci to implement registration/authentication, encrypted job receipt, listener/worker behavior, arbitrary shell/JavaScript/container/composite action semantics, workflow commands, secret masking, logs, artifacts, cancellation, and compatibility with GitHub updates on Linux and Windows. The published Go scale-set client provides controller integration, not a substitute job worker. A simple workflow bridge that invokes ra8ci from an existing runner is a useful migration step, but does not let ra8ci own runner capacity or lifecycle. The selected design gives native GitHub UI/job behavior and lets ra8ci own Proxmox capacity, task semantics, analytics, and hardware safety without maintaining a private Actions protocol implementation.

Demand source, revised. Owning the runner binary stays out of scope; what changed is how ra8ci learns a job exists. The scale-set client is public preview with no fallback in the original design, so a single preview package gated all dispatch. The webhook and just-in-time route uses generally available surfaces, maps directly onto the durable state machine, and costs exactly one new component, the unclaimed-runner reaper. The trade is the loss of the listener's push efficiency and the acceptance of webhook delivery delay, both bounded by the reconciliation pass. Self-hosting a forge was evaluated and rejected for this problem: GitLab's custom executor is in maintenance mode and brings a full second platform to operate, Forgejo and Gitea move the same integration seam rather than removing it, and no evaluated forge supports this design better than the one already in use. A forge migration would be a source-control decision, made for its own reasons, not a response to one preview interface.

Migration with proof at every boundary

The planner AI decides issues, MR batches, and ordering. This is the approved technical dependency spine, not a planner-defined stack.

1. **Executor and evidence.** Define task contracts, local CLI, server run ingestion, deadlines, logs, and the run/task/step schema. Initially delegate to existing commands so comparable measurements start early. No old/new executor may run the same responsibility concurrently.
2. **Board lease.** Move bench.sh semantics into the server, add durable queue and audit, then delete the script and repoint its just recipes in the same MR. Prove cooperative checkpoint and recovery behavior before hardware use.
3. **Dispatch and GitHub capacity.** Introduce the internal demand-source interface first, then implement and shadow-run `workflow_job` intake: verify signatures, deduplicate deliveries, persist delivery order, and reconcile against the Actions jobs API. Take just-in-time registration to a disposable Linux

guest first, then Windows, whose longer boot and provisioning time exposes stale-demand and cancellation races. Ship the unclaimed-runner reaper in the same batch as the first credential mint, and put bench-aware admission ahead of minting so scarce Windows capacity is not consumed while the board is unavailable. Enroll agents in the guests, start an official ephemeral runner with that configuration, and replace both Proxmox CI dispatch scripts with one ra8ci-owned lifecycle that invokes Terraform and Ansible. Prove label matching, one-job teardown, identity-checked cleanup, result propagation, cancellation, failed provisioning, GitHub and server reconnect, and restart reconciliation. Keep the current ARC/Docker/HIL fleet until matching gates are green; do not route the same job class to two competing dispatch paths during cutover.

4. **Absorb checks.** Replace each used script's behavior, repoint every caller, delete the old script in that MR, and require a non-skipped gate. Prioritize frequent/slow checks using real task-step data.
5. **Entry points.** Hooks, workflows, and just recipes call stable ra8ci task names. GitHub workflows may remain thin native Actions entry points: `runs-on` selects the ra8ci scale set, while required setup or third-party `uses:` actions still execute in the official runner. Keep required Zig build-parity and analysis coverage; only retire CMake-facing paths when 857's acceptance evidence is complete.

A migration ledger maps old path, callers, new task, behavior proof, deletion MR, and fallback policy. A task is not "ported" because ra8ci shells out to the old script. The final state must preserve exact exit semantics, artifacts, selftests, and safety checks.

Second migration: ra8ci posts its own check runs

Direction, not authorized work. The end state worth building toward is that GitHub holds the pull request, the review, and the merge gate, while ra8ci publishes one check run per catalog task through a GitHub App and branch protection requires those names. The forge then sits behind a very small adapter: repository events in, check-run state out.

The repository's own workflows make that unusually cheap here. Measured on dev for this revision, `.github/workflows` holds 10 files and 1,566 lines defining 39 jobs, with 39 `runs-on` declarations, one `matrix`, two `needs` edges, ten concurrency groups, about 51 `uses:` steps, and no reusable workflow definitions; `firmware.yml` alone accounts for 26 of the jobs. Actions is expanding almost nothing. It is a flat fan-out of job names the task catalog already owns, which is exactly the case where owning the checks is cheap rather than a rewrite of a workflow engine.

What would actually be rebuilt is the third-party `uses:` steps, rerun controls, log tailing, artifact browsing, and a stable required-check naming scheme. Take this as the second migration, after the demand-source change is proven in production, and adopt it only when the reviewer experience is at least as good as the one it replaces. Run it in shadow mode first and compare conclusions against Actions over representative pull requests before any required check moves.

Verification gates

Unit tests cover state transitions, SQL constraints, CLI parsing, scheduling, auth, deadlines, and Windows/Linux process-tree termination. Property and concurrency tests cover board uniqueness, queue ordering, idempotent retries, and lost acknowledgments. Integration tests use fake agents, disposable databases, and a mocked GitHub demand adapter; they must cover duplicate and out-of-order webhook deliveries, a job cancelled after its credential was minted, and a reservation that reaches its unclaimed deadline. An isolated GitHub/Proxmox acceptance run proves real webhook delivery and signature validation, just-in-time runner registration, one Linux job, one Windows job, cancellation, cleanup, reaper expiry, and recovery from controller restart; it is not a full CI sweep. Emulator tests precede the single matching hardware test. Hardware work requires a live lease and stops at the next safe checkpoint on yield. Full CI must be green; warnings and skipped tests are not passes.

Implementation appendix

Internal Go boundaries

Keep one executable but explicit packages: catalog defines task schemas and validation; cli handles human input and machine-readable output; server owns authentication, API, transitions, scheduling, and reporting; github adapts the Actions Runner Scale Set Client behind a narrow capacity interface; agent owns bounded process execution and resource sampling; provision defines a narrow Terraform/Ansible interface; board owns lease policy; store owns SQL transactions and migrations. Dependencies point inward toward task and state contracts. Provisioning code must not call board hardware directly, GitHub job payloads must not become provisioner commands, and agents must not import server database packages.

A task implementation has prepare, execute, checkpoint, and finalize phases. Prepare is side-effect-free where possible and validates tool availability, repository state, arguments, and output paths. Execute emits typed step boundaries. Checkpoint is the only place a yield can be honored before more board I/O. Finalize writes artifacts and releases resources even after task failure. Every phase has an idempotency key derived from run, task, attempt, and phase.

API contract sketch

- POST /v1/runs accepts trigger, immutable source identity, requested task set, and a client idempotency key; returns a run ID.
- GET /v1/runs/{id} returns state and attribution timestamps. GET /v1/runs/{id}/events streams ordered, resumable progress by event sequence.
- POST /v1/runs/{id}/cancel records intent; it does not falsely report an immediate process stop.
- POST /v1/agents/enroll exchanges a short-lived bootstrap credential for a capacity-reservation-bound guest identity; only a later, verified job grant is run-bound. The persistent board agent has a separate operator-enrolled host identity.
- GET /v1/agents/{id}/assignments is an authenticated long poll. The agent acknowledges an assignment with its attempt ID before execution.
- POST /v1/attempts/{id}/chunks accepts numbered log or resource batches and is idempotent on attempt plus sequence.
- POST /v1/attempts/{id}/result accepts a terminal receipt with exit code, deadline outcome, artifact digests, and final sequence.
- POST /v1/boards/{id}/requests creates a waiter. GET /v1/boards/{id} returns holder and queue. Release, extend, and yield endpoints require the appropriate lease generation and role.
- GET /v1/reports/slow returns a window, filters, sample counts, quantiles, and exclusion reasons.

All mutating endpoints accept an idempotency key and return a correlation ID. API versions are explicit. A protocol mismatch is a visible failed enrollment or drained agent, not an undefined task result.

Transaction and lease sketch

A board has a single current generation. Granting a lease locks its board row and requires recovery_complete for the current generation, regardless of whether its former lease expired. It then chooses the next eligible waiter, increments the generation, inserts a grant_pending lease, changes waiter state, and appends audit events in one transaction. Only after the board agent durably installs and acknowledges the generation does the lease become active and usable. A uniqueness constraint on pending or active board ID is defense in depth. Heartbeat and extension compare generation and expiry; a stale holder cannot extend. Release compares the same token and records the end reason. If neutral state is not confirmed, recovery_required remains true and the next grant is blocked.

The board-side agent must validate the token at a checkpoint and immediately before a board-touching command. A token check is not sufficient on its own for a long electrical operation, so each such

operation has a declared maximum safe segment and a local deadline. The server does not claim a lease is safely reusable merely because PostgreSQL time has passed.

Queue claims likewise use a transaction and a stable attempt ID. If the server crashes after dispatch but before receiving acknowledgment, the recovered server asks the agent for that attempt ID; it does not issue the same non-idempotent task twice. If the agent is gone, the attempt becomes lost and a policy-controlled fresh attempt may be created.

GitHub scale-set handoff contract

The GitHub-facing adapter is a capacity controller, not a job executor. It persists each scale-set demand/decision with an idempotency key and a requested OS/capability profile. The provisioner creates one approved guest identity and returns a verifiable VM marker. After guest health and agent enrollment, the controller requests a short-lived JIT runner configuration and delivers it to that guest only. The official runner registers, accepts at most one GitHub job, executes its steps, and reports native job status. ra8ci separately correlates the GitHub workflow run/attempt/job IDs with its own run and task attempts. The order of GitHub assignment and ra8ci telemetry receipt is not assumed; missing correlation remains visible.

The controller records busy/idle state before any drain or destroy. A GitHub cancellation requests a graceful job stop; a timeout follows the runner's cancellation semantics and ra8ci's local task safety policy. A board operation never inherits permission to hard-kill an indivisible flash merely because the outer GitHub job ended. After the worker exits, ra8ci reconciles GitHub registration, agent result, VM marker, and Terraform state, then tears down the guest with identity checks. Lost acknowledgment or failed destroy becomes `cleanup_failed`, not success. A controller restart reconstructs this state before requesting new capacity.

The first acceptance matrix covers Linux and Windows JIT registration; `runs-on` matching; one job per guest; ordinary `run:` and representative `uses:` steps; log and conclusion parity in GitHub; cancellation while idle and busy; token expiry; failed guest boot; controller/database outage; cleanup failure; and no Proxmox/database credentials in the worker. The public-preview client is pinned and tested before any old runner lane is removed.

Measurement specification

Use monotonic clocks for durations within one process and UTC timestamps for cross-host correlation. Record both, plus clock-skew diagnostics. Top-level run phases are mutually exclusive intervals on the server clock; nested task and board spans are linked to their parent phase and never summed into the parent wall time. A step's duration uses its agent's monotonic clock. Step timing cannot be computed by subtracting clocks from different hosts without a synchronization assumption. Host facts at task start satisfy provenance; periodic process-tree samples support resource analysis. Sampling interval and dropped samples are recorded, so low observed CPU does not masquerade as measured low CPU.

The first performance hypothesis should be tested as follows: select tasks with at least a minimum sample count on comparable host and engine profiles; calculate median and p90 execution and queue time; inspect CPU-seconds divided by wall seconds, peak RSS, I/O and cache state; simulate an additional concurrency slot against declared memory/CPU budgets; then trial a bounded change and compare critical-path time and failure rate. Revert if p90, memory pressure, or flakiness worsens. Do not move a required gate to optional as a shortcut.

Yield estimator and replay contract

At each human request, record the target holder's task definition/version, image digest, board/fixture revision, current phase, request time, first safe checkpoint time, neutral/available time, and whether cancellation, recovery, or an overrun intervened. Human ETA uses request-to-neutral/available, not only request-to-checkpoint. Estimate only from comparable, completed samples. The first policy uses a 60-day rolling cohort, at least 30 samples, and nearest-rank p95 plus max(5 seconds, 20 percent);

it is floored at the declared remaining safe segment plus restore/probe bounds. Report sample count, censored/failed count, and overrun rate. A maximum observed task runtime plus standard deviation is not a sound yield estimator: it measures different work, is sensitive to one outlier, and hides censored or failed attempts. If data are sparse or stale, use the declared segment plus restore/probe bounds; if a board task has no bound, reject automatic dispatch and show ETA unknown. Thirty seconds is a handoff target, never a safety deadline. Never let learning override a physical safety limit or an existing HIL validity deadline.

Each board task declares cancellation class (checkpointed, safe process cancellation, or noninterruptible), safe-step bound, retry class (idempotent, restart from baseline, or manual), and restore action. Persist the command/input snapshot and content-addressed firmware image, but do not pretend JSONB can snapshot running RAM or peripherals. A replay after preemption uses a new attempt ID and reacquires a lease; it verifies the board and flashes its recorded image before restarting from the beginning, unless a tested task-specific checkpoint explicitly permits more. Repeated preemptions and failed restores are separate outcomes in analytics, not successful short yield samples.

Deployment and recovery drill

Package one versioned ra8ci binary per target OS/architecture: Linux amd64 for the control VM and Linux guest, Windows amd64 for the Windows guest, and Linux arm64 for the Raspberry Pi 5 board agent. A reviewed systemd unit starts the server in the protected control VM with an unprivileged identity; PostgreSQL is colocated and reached through a local Unix socket. Only a restricted provisioner identity reaches approved Proxmox operations. Disposable guests receive both an official ephemeral GitHub runner and a ra8ci agent whose initial enrollment and lifetime are bound to a capacity reservation, not an as-yet-unknown GitHub run. On Windows the agent runs as a managed service with a deliberately narrow account. The Proxmox host itself runs neither repository code nor the GitHub job worker.

Before production use, perform restore drills for PostgreSQL and artifact storage; server restart during provisioning; agent disconnect during Zig build; server and database outage during a board flash; expired lease with an unresponsive board agent; lost cleanup acknowledgment; and Windows process-tree cancellation. Each drill has a recorded expected state and an operator recovery command. The system is ready only when it reports incomplete work truthfully and refuses unsafe automatic cleanup or board reassignment.

Executable implementation contract

This section fixes first-pass software defaults. It does not authorize a live Proxmox change, a GitHub App installation, a branch-protection edit, a board operation, or a script deletion without its behavior proof. The implementation branch is `ci/ra8ci-implementation`, cut from `origin/ci/orchestrator` at `7ff60fee72cbc0d8e2754de9cff247fa5c24ee46`. The separate planner still owns issue and MR boundaries; this document defines engineering acceptance, not a stack.

Repository layout and task migration

The Go module is `tools/ra8ci/go.mod` (module `github.com/bsikar/ra8-firmware/tools/ra8ci`), compiled as the `ra8ci` command for Linux amd64, Linux arm64 (the Raspberry Pi 5 board agent), and Windows amd64. Source packages are `internal/catalog`, `internal/cli`, `internal/server`, `internal/agent`, `internal/store`, `internal/github`, `internal/provision`, `internal/board`, and `internal/evidence`; their imports follow the boundaries in the preceding appendix. The command has `server`, `agent`, and `task/board/report` modes. Go 1.27.1 is already pinned by the devcontainer; the GitHub scale-set dependency is pinned to v0.4.0, not latest.

The reviewed catalog source is `tools/ra8ci/catalog/tasks.json`, schema version 1. It is embedded in the binary; `tools/ra8ci/catalog/sha256.txt` records the SHA-256 of canonical JSON (sorted keys, UTF-8, no insignificant whitespace). Each task record has `name`, `version`, `tier`, `scope`, `os`, `capabilities`, `args_schema`, `deadline_seconds`, `board_policy`, `steps`, `outputs`, `retry`, and `resource_hints`. Unknown fields, unknown task names, invalid arguments, digest mismatch, and unsupported schema fail closed. There are no compatibility adapters in v1. Pinned binary/package checks belong to deployment and CI; the app does not invent a second release-signing system.

The Phase 0 inventory and work-unit JSON are the exhaustive migration input. The migration ledger `tools/ra8ci/catalog/migration.json` has one row per in-scope used file: `old_path`, `callers`, `new_task_or_package`, `old_behavior_proof`, `new_behavior_proof`, `deletion_commit`, and `state`. A file with no stand-alone entry point maps to a Go package, not a fictitious CLI task. A migration first records golden success, failure, timeout, output/artifact, and selftest behavior from the old implementation; then runs the Go replacement against the same fixtures; then repoints every caller and deletes the old file in one change. Shelling out to that old file is an executor-stage adapter, not a completed port. No skipped or warning-bearing gate counts as proof. The planner's existing work units decide the eventual MR boundaries.

The first executable slice is local executor and evidence for four tasks, without board, Proxmox, or GitHub deployment. All four have catalog schema 1, no retry, one named command step, no board policy, and no output artifact beyond logs/receipt: `format` runs `bash scripts/checks/format_tree.sh`, is `safe-local/write-working-tree`, and has a 900-second deadline; `format-check` runs that script with `--check`, is `safe-local/read-only`, and has a 900-second deadline; `lint-go` runs `just quality::local::gate lint-go`, is `safe-local/read-only` with a 1200-second deadline; `test-go` runs `just quality::local::gate test-go`, is `safe-local/read-only` on Linux with a 1800-second deadline. Arguments are empty for all four. Each uses the verified repository root as working directory, inherits a scrubbed allowlist of environment variables, records exact exit and step duration, and fails if its tool is missing. No in-tree build output is allowed. Acceptance is equivalence of pass/fail and formatter diff on representative clean/failing fixtures plus 85-percent Go coverage/race gates.

The full product is not accepted at that slice: board control, GitHub capacity, provisioning, reports, and every inventory used helper remain required work. Subsequent slices follow the engineering dependencies in the architecture and the planner's work units. `build`, `emulator`, and `hil` become semantic entry points only when their Zig/out-of-tree build, emulator parity, and board safety contracts are available; a task name never silently becomes a no-op. The catalog may temporarily invoke the exact existing just recipe or `scripts/ci.sh --gate` under a controlled working directory while measurements accumulate. The current `RA8_GATE_REGISTRY` and CI-parity gate remain authoritative until each gate is

absorbed. Zig steps are introduced only as 857's build-graph acceptance makes them available; ra8ci pins the Zig version, target, step, out-of-tree cache and artifact digest in each task catalog revision. It does not implement a parallel build graph. A generated per-task mapping is reviewed against the current justfile, imported just/*.just, hooks, and workflows before any call site moves.

Internal result and CLI contract

Internal packages return a typed `Error` with code, operation ID, retryability, and cause. Codes are `invalid_argument`, `denied`, `conflict`, `stale_generation`, `expired`, `unavailable`, `deadline`, `cancelled`, `incomplete_evidence`, `unknown_external_outcome`, and `protocol_mismatch`; callers never infer retryability from message strings. `provision` returns unknown outcome after lost acknowledgment and demands reconciliation before retry. `store` reports failed commit as unavailable. `board` reports stale generation or recovery required. The server maps these to `application/problem+json`, and the CLI prints a short message or versioned JSON.

CLI exit categories are 0 success, 1 task failure, 2 usage/invalid argument, 3 unavailable, 4 denied, 5 deadline, 6 cancelled, 7 incomplete evidence, and 8 protocol mismatch. The exact child process exit code lives in the task attempt and `--json`; numeric child status is not overloaded as an orchestrator transport error. `ra8ci task --local` is allowed only for catalog tasks marked `safe-local`; `--dispatch` requires healthy server, audit, and database. `ra8ci run status|logs|cancel|retry|explain`, `board take|release|extend|status|queue|doctor|recover|quarantine`, and `report slow|critical-path|resource-fit|board|runners` are versioned command families. Human and JSON output expose the same result kind and evidence completeness.

Identity, network, and credentials

The server ignores any actor or priority supplied in a request body. Human v1 identity is an operator-issued short-lived mTLS client certificate mapped to observer, submitter, board-human, or operator role in PostgreSQL; the client stores its key in protected user storage. Role grants are scoped by repository ID and optional board ID. An observer reads nonsecret run/report/board status only for granted repositories/boards. A submitter creates runs and may cancel/retry only its own runs within a granted repository; it cannot use human board priority. Board-human can request, extend, and use a named board but cannot cancel another holder or invoke recovery. Operator can inspect audit, quarantine/recover, drain, and cancel any scoped run with a reason; operator actions never bypass hardware neutrality. GitHub App, guest, and board-agent service principals have separate minimal scopes and cannot impersonate a human. Every read and mutation checks scope before data access and audits denial. The persistent board-side agent uses a distinct host certificate, an operator-approved enrollment and renewal path, and revocation checked on reconnect. A disposable guest receives a one-use enrollment token bound to a capacity reservation, exact VM marker, host class, and short expiry. Before GitHub job correlation it can report only health; after correlation, the server issues a separate task-scoped grant. Neither guest nor board agent receives database, GitHub App, or Proxmox credentials. Certificate issuer, approved principals, and rotation procedure must be named before production.

The control VM has an approved management address and no direct listener on the isolated guest bridge. A narrow L4 relay on a lab gateway may forward guest agent TLS to the control VM without terminating mTLS; an outbound proxy permits the official runner to reach GitHub's required domains and reviewed dependency endpoints. Guest firewall denies the management subnet, Proxmox API, PostgreSQL, and other guests. Provisioner-only SSH/WinRM is allowed through a constrained gateway. The specific bridge, relay IP, route, DNS/NTP service, and workflow egress list are operator-approved inputs, not inferred from reachable personal networking. An isolated Linux and Windows acceptance test must prove both permitted execution and forbidden management access.

The protected control VM and colocated PostgreSQL use separate unprivileged Unix identities and a database Unix socket. The runtime database role has no DDL and no UPDATE/DELETE on `audit`; a migration role alone changes schema. A dedicated scoped Proxmox API credential and a repository-

scoped GitHub App key live only on this VM. No repo-controlled code or official runner executes there or on the Proxmox host. Until credential delivery and off-VM backup are approved, server/provisioner integration is tested with fakes and disposable local databases only.

API and offline evidence

The `/v1` API requires TLS and an authenticated principal. Every mutation has Idempotency-Key (UUID) scoped to principal, method, path, and canonical body SHA-256. Exact replay returns the prior response; same key with different body returns 409. Errors use `application/problem+json` with code, message, request_id, retryable, and nonsecret details. Map invalid argument to 400, unauthenticated to 401, forbidden to 403, missing to 404, conflict/stale generation to 409, expired to 410, rate limit to 429, protocol mismatch to 426, and unavailable to 503. No endpoint accepts arbitrary shell text or an unreviewed path as a privileged argument.

`POST /v1/runs` accepts trigger, immutable source {repo, commit, snapshot_sha256}, catalog digest, task records {key,name,args,depends_on_keys}, and optional parent run ID. Keys are unique within a run; dependencies must name present keys and form an acyclic graph. The catalog separately declares dependencies among steps. It returns run ID and queued state. A failed, cancelled, timed-out, or evidence-incomplete prerequisite marks descendants skipped with a reason and makes the run non-success; skipped work is not counted as a pass. `task_edges` is populated from these validated task and step declarations. GitHub workflow-level edges are included only if an exact static needs graph from the pinned workflow commit can be mapped to observed jobs; otherwise `critical-path` labels the GitHub segment partial and reports no invented full-run path. `GET /v1/runs/{id}` includes execution result, cleanup status, evidence completeness, phase intervals, and task/attempt IDs. `POST /v1/runs/{id}/cancel` records intent and returns 202, not a claim of process termination. `GET /v1/runs/{id}/events?after=N` streams SSE with monotonically increasing per-run sequence, 15-second keepalive, Last-Event-ID resume, and an explicit retention-gap error. Reads are authorized by repository/run role and paginated with opaque cursors.

For direct `--dispatch`, the client snapshots its checkout into a deterministic archive containing tracked files and recursively pinned submodule content, rejects untracked/dirty content in v1, and hashes the complete archive. `POST /v1/source-staging` under the submitter identity reserves a one-use upload ID and bounded size/TTL; `PUT /v1/source-staging/{id}` streams the archive and verifies digest, length, path safety, and source metadata before atomic promotion. `POST /v1/runs` binds exactly one staged upload ID to the same principal, repository, commit, and digest in its admission transaction; an expired, reused, or mismatched upload fails. The target agent downloads and verifies size/digest, rejects absolute paths, `..`, and escaping symlinks, extracts into a per-attempt root, and verifies the embedded source commit and catalog before executing. GitHub jobs instead use the official runner's checkout; they do not accept a client-provided source tar as identity proof. Missing source evidence blocks dispatch, not a fallback clone of a mutable branch.

`POST /v1/agents/enroll` exchanges a one-use reservation-bound token, VM marker, version range, and host facts for a scoped identity. `GET /v1/agents/me/assignments?wait=25s` long-polls. `POST /v1/assignments/{id}/ack` echoes attempt ID and assignment version. A 5-second heartbeat reports current attempt, phase, host facts, and clock offset and returns cancel/yield intent. Assignment acknowledgment and terminal receipt are idempotent on stable attempt ID. A server absolute deadline is sent as remaining time; the agent enforces a monotonic local limit. Linux and Windows process-tree termination use bounded graceful then hard-stop phases, except board operations, which obey their declared indivisible segment and recovery contracts.

`POST /v1/attempts/{id}/chunks` accepts stream, sequence, step ID, monotonic offset, at most 64 KiB of bytes, and SHA-256. Duplicate sequence and digest succeeds; duplicate sequence with different digest conflicts. Backpressure causes a bounded encrypted agent spool, never silent log loss. Reconnect resumes from the last contiguous acknowledgment. `POST /v1/attempts/{id}/result` carries exact child exit, typed result, final sequence per stream, step summaries, and artifact digests. The attempt is

not evidence-complete until all chunks and referenced artifacts are durable. `PUT /v1/artifacts/{sha256}` is attempt-scoped, length/digest checked, streamed to a temporary file, flushed, and atomically promoted. `GET /v1/artifacts/{sha256}` requires a task or board-agent grant. Board commands use lease ID and generation; reports include cohort, sample count, exclusions, and timing provenance.

`POST /v1/local-receipts` accepts the canonical receipt plus digest under a submitter identity and returns upload ID and `local_unverified` state; the server never turns it into a CI attestation. `GET /v1/runs/{id}/logs?after=N`, `GET /v1/reports/{slow|critical-path|resource-fit|board|runners|idle-time|regress}`, and board `request/status/release/extend/yield/doctor/recover` endpoints use the same versioned error envelope, role checks, opaque pagination and idempotency rule. Every board mutation includes board ID, lease ID, generation, reason, and a monotonic request ID; release and extension require the current holder, recovery requires operator role, and report filters are parsed typed values, not SQL fragments. The first report slice is `slow`, `board`, and `runners`; `idle-time`, `regress`, and full critical-path attribution follow only when the required samples and edges exist. No advertised report prints invented values when evidence is insufficient.

Safe local work writes a versioned canonical JSON receipt under `$XDG_STATE_HOME/ra8ci/spool` on Linux or `%LOCALAPPDATA%/ra8ci/spool` on Windows with a restrictive owner ACL. It includes receipt UUID, source/catalog/binary digests, task and argument digest, local-unverified actor/host, UTC endpoints, monotonic elapsed, exact child exit, step summaries, and log/artifact digests. Temp-write, file flush, atomic rename, and directory flush where supported make it durable. Corrupt/full spool is an explicit incomplete-evidence result. Upload by UUID and digest is idempotent; collision quarantines the receipt. A synced offline receipt remains `local_unverified`, never a CI gate attestation.

Durable state and measurements

Use UUIDv7 for server run, task, attempt, reservation, and external-operation IDs; use UTC `timestampz` for cross-host events and `bigint` monotonic offsets for local durations. Store run, task, attempt, step, dependency edge, resource sample, artifact, log chunk, board waiter/lease/session, board row, agent, GitHub job, runner lifecycle, external operation, idempotency key, run event, and audit tables. Primary identity/state is typed; JSONB is only versioned optional metadata. Unique keys include `(task_id, attempt_no)`, `(attempt_id, stream, seq)`, `(run_id, event_seq)`, guest marker, operation ID, and principal/method/path/idempotency key. Queue, sample, lease, and runner-state indexes follow their query keys. The board has generation and recovery-required columns plus a partial unique index for one pending/active lease per board. Every state transition uses version compare-and-swap and writes its audit/event row in the same transaction. Forward-only migrations run with a separate role; startup refuses an incompatible schema or unsafe downgrade.

Run phases are accepted, queued, provisioning, ready, running, draining, cleaning, then terminal. Execution result is separate from cleanup result so a successful command followed by an orphan guest is `cleanup_failed` overall. Task state is scheduled, assigned, running, terminal. Attempt state is issued, acknowledged, running, then succeeded, failed, `timed_out`, cancelled, preempted, or lost. A retry gets a new attempt ID and never overwrites the old one. Runner lifecycle is requested, provisioning, healthy, JIT issued, registered idle, busy, draining, deregistered, destroyed, with reconcile-required side state. Commit external intent, exact expected identity, and operation ID before side effect; after lost acknowledgment inspect state, Proxmox marker, GitHub runner, and agent facts before retrying.

Native ra8ci queue admission caps outstanding requests at 100 per profile, returning retryable 429 above the cap. Required tasks outrank optional, which outrank nightly; FIFO orders each tier. Every full minute waiting moves an optional/nightly task up one effective tier, capped at required, but it does not interrupt a running task. Host caps come from the reviewed fleet declaration and free-RAM/disk checks; no overcommit is inferred from low instantaneous load. If a profile has at least two slots, one is reserved for queued required tasks with predicted duration at most 10 minutes; the slot may be borrowed when no such task waits and is returned only at a task boundary. Quiet-hours denial is

an admission condition, not a guessed host fact. GitHub owns its workflow queue; the separate ra8ci capacity controller never attempts to reorder its jobs.

Process-tree samples occur every 5 seconds and at step boundaries. Record actual interval, dropped count, CPU time/utilization, process-tree RSS/peak, load, free RAM, disk and network I/O, agent monotonic offset, server receive time, and clock-skew estimate. Cross-host durations never subtract unsynchronized clocks. Raw logs, firmware, and other artifacts use SHA-256 content-addressed files on an encrypted persistent control-VM volume; PostgreSQL stores metadata/pointers. Redact secrets before persistence. Draft retention is 30 days raw logs, 90 days samples/artifacts, one year run metadata, two years audit, and board baseline images while referenced; incident evidence is pinned. Reports show `insufficient` data for p90 when fewer than 20 comparable attempts exist and include failures/exclusions, host, engine, profile, cache state, catalog version, and timing uncertainty. Tier changes remain human-reviewed.

GitHub scale-set and required result

Use `github.com/actions/scaleset v0.4.0` at tag commit `6ce025902cd964747a078c2aabe7340ebc667eca`. The first isolated prototype uses repository scope `https://github.com/bsikar/ra8-firmware`, single-label sets `ra8ci-linux-amd64` and `ra8ci-windows-amd64`, minimum idle 0 and maximum one guest each. Production labels, runner group, quotas, image digest, and runner release require review. The App needs repository Administration write, Actions read, Checks write, and Metadata read; installation is not implied. `TotalAssignedJobs` drives desired capacity, and honest maximum-capacity reporting is bounded by approved Proxmox budgets. Provisioning, idle, and busy guests count against capacity; busy workers are never destroyed for scale-down.

The v0.4.0 listener acknowledges messages before invoking scaler callbacks. Wrap its message client so normalized message ID, session, statistics, and required job fields commit to a durable inbox before acknowledgment; callbacks commit idempotent outbox intent, not external side effects. Replay inbox/outbox after restart and reconcile GitHub runner list, Terraform state, Proxmox marker, and agent receipts. Unknown message types or canceled/requeued assignments trigger REST backfill, not assumed success. A capacity reservation binds guest VM ID/marker and Terraform state path before guest enrollment. An idle guest grant permits only health. Bind runner ID/name and reservation to trusted GitHub job events, then use the paginated `workflow-jobs` REST API to verify repository, workflow run, attempt, job ID, runner request/runner identity, and head SHA before issuing any task grant. Missing or contradictory correlation remains unverified and blocks positive results.

The official runner owns the GitHub job conclusion. Before issuing JIT, the controller uses Actions-read REST polling of queued **jobs** in queued or in-progress workflow run attempts, including a downstream needs job in an already-running workflow, to discover repository, run ID, attempt, head SHA, job ID, and target scale-set label. It holds a capacity reservation without JIT if this lookup has not converged; it does not need a public inbound webhook. A single App check named `ra8ci / <workflow> / lifecycle`, keyed by workflow run/attempt in `external_id`, is created in progress on that head SHA before a matching guest can take work. All jobs in that workflow attempt that target ra8ci sets are recorded from the job listing, including not-yet-ready needs descendants; the check remains pending until the workflow attempt is terminal and every such job's assigned runner, agent evidence, and guest cleanup has a verified terminal outcome. A newly discovered matrix job extends the set while pending. Missing job correlation or cleanup makes the aggregate fail, never silently pass. On same-SHA concurrent runs, each check has a distinct external ID; the prototype must prove the repository's required-check semantics before this check becomes an authoritative branch gate.

Failure, cancellation, timeout, missing evidence, and cleanup failure make the separate lifecycle check fail; the native job check cannot be retroactively changed. The protected branch requires the lifecycle check from the ra8ci App in addition to native checks only after acceptance. An uncertain Checks API write is reconciled by listing check runs for SHA/name/App/external ID, not blindly repeated. If queued-job discovery or pending-check publication fails, advertise zero new capacity. Reruns and concurrent

runs on one SHA are explicit acceptance cases. No protected-branch rule is changed without owner approval.

Inside the one-job guest, the official runner's `just` recipe calls the local `ra8ci` CLI through a Unix socket or Windows named pipe served by the agent. OS peer identity must be the runner service account; a caller-supplied GitHub environment variable is only a hint. If the server's verified job grant has not arrived, the CLI waits at most 60 seconds while the agent refreshes correlation, then fails closed; cancellation interrupts the wait. The server's reviewed policy catalog maps (repository ID, trusted workflow path/ref, job name, event class, runner scale set) to an explicit set of allowed task names and argument schema versions. It is embedded in the server binary, never taken from PR-controlled workflow YAML; fork PRs and untrusted refs cannot receive board or provisioner capabilities. A HIL task additionally requires the dedicated HIL workflow/job identity and board lease, not merely an agent label. The agent accepts only those catalog tasks under a verified job grant, executes as a nonprivileged task identity, and associates logs/results with the server-issued attempt. It verifies the canonical `GITHUB_WORKSPACE` path lies under the runner's dedicated work root and its checked-out commit matches the verified GitHub head SHA; it rejects symlink escape and mismatches. Job completion revokes the grant. A background workflow process cannot gain a new grant after the one job ends. A compromised guest can still falsify its own workspace or availability, so its reports are labeled guest-origin evidence, not a cryptographic proof of firmware correctness; the disposable boundary protects management credentials and later jobs.

Board, artifact, and physical recovery

The persistent Pi board agent is the sole software executor for J-Link, UART, reset, relay, and power actions. Restrict SSH and device ACLs so disposable guests and legacy scripts cannot touch these devices directly after cutover. It owns child process groups and a local serialized gate, persists its highest installed generation, and checks lease generation/expiry both at enqueue and immediately before each step. A DB grant starts as pending and becomes usable only after the agent persists and acknowledges that generation. Expiry or server loss forbids a new segment but lets an already-started, locally bounded indivisible flash finish; no next grant occurs until quiescence and an approved neutral receipt. A restored database generation below the agent's high-water mark forces quarantine. Physical/root/direct-probe access remains outside software fencing.

A board image arrives through the authenticated artifact endpoint with immutable digest, size, source/catalog identity, and approved flash arguments. The board agent downloads, verifies the full unstripped image before preparing any stripped flash form, and rejects corruption or missing baseline. A versioned fixture-specific neutral profile checks supervised process/fd quiescence, unexpected device owners, expected relay/power/reset states, probe and board identity, VTref/sensors, and baseline application health where available. `probe.sh` connectivity alone is not neutral proof. If the fixture lacks observability or restore, quarantine for operator inspection. The operator must approve baseline digest, permitted reset/erase/power actions, physical-clear procedure, and external peripheral profile before live use.

Initial heartbeat is 5 seconds; after 15 seconds without board-agent heartbeat, new commands stop. Human default duration is 30 minutes, CI per-app default 15 minutes and renewable under a two-hour suite ceiling, AI default 10 minutes renewable to one hour. No segment starts without remaining lease time at least its declared bound plus recovery margin. Human priority exceeds CI then AI; FIFO within class and no new lower-priority board step starts while a human waits. A second human never force-preempts the first. Extension is reasoned and bounded, with at most a 10-minute safe-wrap-up extension while another human waits. An eight-hour exceptional cap requires operator reason and physical confirmation. HIL `hil.conf` validity windows are separate from flash bounds and yield estimates. A task with no declared max segment/restore policy cannot be automatically dispatched to the board.

Board state is ready, grant_pending, active, yield_requested/draining, recovery_required, recovering, or quarantined. board doctor is read-only; board recover --plan <reviewed-id> --why runs only an allowlisted recovery plan; board acknowledge-ready --probe <receipt-id> --physically-clear --why requires operator authority where visual inspection is needed. board force-release only invalidates the old holder and starts recovery. The yield estimator uses 60 days of comparable full request-to-neutral latency, at least 30 successful samples, nearest-rank p95 plus max(5 seconds, 20 percent), floored at remaining safe-segment plus restore/probe bounds. Sparse history uses the declared bound; absent bound means unknown ETA and no automatic run. Thirty seconds is a handoff target, not a flash timeout.

Lease tokens carry board ID, generation, server UTC expiry, and nonce bound to holder identity. On receipt the board agent requires synchronized UTC with a measured worst-case offset bound at most two seconds; it seeds a local monotonic deadline as $\max(0, \text{server_expiry_utc} - \text{local_utc_now} - \text{offset_bound} - 2s_safety_margin)$. Network delay is already reflected in local UTC at receipt and can only shorten the budget; no assumed one-way transit time grants authority. Every heartbeat can shorten but never lengthen the local deadline; an authorized extension creates a separately audited higher-version deadline after a fresh offset check. Unbounded clock error, heartbeat loss, agent reboot, or monotonic continuity loss forbids starting a new segment and enters recovery-required. A running indivisible segment obeys its previously declared local hard bound, not wall-clock jumps. The agent stores generation high-water and last deadline state before acknowledging a grant. No expired lease or restored database clock can extend authority.

A human holder uses `ra8ci board flash <digest>, probe, reset, serial attach, OR debug start|stop` with a live lease; those commands become allowlisted agent operations rather than raw SSH/device access. UART and debugger sessions are proxied through the authenticated server to the board agent, bound to lease generation and a bounded local process group. A human can request a graceful stop or checkpoint; the agent ends its debug/serial session and verifies the fixture before release. The first live board profile is not inferred from a J-Link probe: the operator must approve the exact baseline firmware digest, fixture neutral evidence, permitted reset/erase/power actions, flash arguments, and physical-clear procedure. Until that profile is committed and tested, board mode stays disabled in production and all board tests use a simulated agent.

Provisioning, operations, and proof

The strict Terraform/Ansible path is the starting safety reference. The direct `qm` server runner and broad Python manager destroy path are not cleanup APIs. Provisioning must preserve template ID/name/stopped state, reserved pool/datastore/bridge, run marker, local Terraform state, VM ownership checks, and stopped-state guard before purge. Use the existing `infra/terraform/environments/lab` configuration with pinned Terraform $\geq 1.10, < 2$, `bpg/proxmox` $\sim 0.83.1$, and Vault $\sim 5.11.0$; existing reserved Linux 9001/template 9000/vmbr9 and Windows 9011/template 9010/vmbr8 identities permit only one prototype guest per profile. The current strict `scripts/dev/proxmox_lab_ci.sh` provides preflight and cleanup behavior, not an API to launch a long-lived GitHub runner; the Go provisioner must extract/equivalently test those guards rather than invoke its full `ci` command. Persist state under a per-run protected directory before apply; capture Terraform plan digest, module/provider versions, inventory, expected VM identity, and cleanup proof. Fixed argument-array calls `run terraform init, validate, plan,` and `apply` with `TF_DATA_DIR` and run-scoped state; cleanup validates exact marker, pool, storage, non-template, and stopped state before destroy, and preserves state on failure. Never replay create or destroy after uncertain exit without reconciling actual state.

New Ansible runner-readiness playbooks under `infra/ansible/playbooks/ra8ci-lab-linux.yml` and `ra8ci-lab-windows.yml` reuse existing lab connectivity and hardening roles but install the pinned official runner and `ra8ci` agent instead of running just `ci` to completion. They receive an exact per-run inventory, source-free image/version manifest, one-use agent enrollment token, and runner JIT config only after health checks; no repository shell string is an extra-var. They return a JSON proof with guest marker, OS/arch, runner/agent digests, clock/skew, disk, network reachability, and service readiness. The Go caller

verifies this proof and source VM identity before JIT issuance. Replace the macOS-keychain OpenBao wrapper with a control-VM credential adapter only after a scoped credential is approved. The control-VM Terraform/Ansible declaration is separate from the disposable lab modules; no existing fixed VM ID is silently reused for it.

After PostgreSQL point-in-time restore, the server starts in reconcile-only mode with zero GitHub capacity and no board grants. It enumerates ra8ci-marked guests in the scoped Proxmox pool, archived Terraform state manifests, GitHub scale-set runners/checks, and agent heartbeats. A guest or external operation absent from restored SQL is quarantined, not adopted or destroyed by ID alone; operator-supervised reconstruction requires matching marker, pool/storage, state digest, and GitHub runner identity. If Terraform state is missing, an operator may reconstruct it with `terraform import` into a new protected run-scoped state, inspect a no-change plan and exact resource inventory, then authorize identity-checked cleanup. If import cannot establish exact identity, the guest is power-isolated and stopped under an audited break-glass procedure, then manually removed only after two-person confirmation of VM ID, marker, pool, storage volumes, non-template state, and absence of a busy runner; wildcard deletion is forbidden. If those facts cannot be proved, quarantine persists and dispatch stays drained. Missing state becomes an audit incident and a failing/pending lifecycle check. Only after every orphan has a verified disposition and board agent high-water/physical state reconciles may an operator lift drain. The restore drill must demonstrate this path, not merely that PostgreSQL starts.

An operator must approve an unused 9000+ control VM ID, management VLAN/address, storage, certificate issuer, relay/proxy path, scoped Proxmox API and GitHub App credentials, and an encrypted off-VM backup target. Initial sizing target is 4 vCPU, 16 GiB RAM, and 250 GiB protected persistent data, subject to measured sizing and Proxmox capacity. PostgreSQL uses local socket only. Draft backup target is nightly base backup plus continuous WAL with 15-minute RPO, daily artifact/Terraform-state manifest backup, 30-day retention, monthly restore to a separate VM, and four-hour RTO. Production waits for an approved destination and a successful restore drill. A restored DB forces board recovery-required until agent high-water and physical state are reconciled.

`/live` reports process life; `/ready` requires database write and audit commit, compatible schema, disk headroom, scheduler health, and backup freshness. Drain stops new assignments and grants but allows safe segments and cleanup to finish. Initial alerts cover backup/WAL lag above 15 minutes, log ingest lag above 5 minutes, agent heartbeat beyond two periods, cleanup stuck 15 minutes after worker exit, disk warning at 80 percent and critical at 90 percent, certificate expiry within 30 days, and orphan guest/board recovery. These thresholds are measured and tuned after pilot operation.

Verification requires race/coverage-clean Go gates, a disposable PostgreSQL concurrency suite for one-lease uniqueness and audit idempotency, fake GitHub/agent lost-ack tests, Linux and Windows spool/process-tree tests, log-gap/corruption tests, and DB/artifact/Terraform-state restore tests. An isolated GitHub prototype runs one Linux and one Windows job, representative `run:` and `uses:` actions, JIT expiry, cancellation, controller restart, guest boot failure, same-SHA rerun, and identity-checked cleanup. Firewall tests prove guest-to-GitHub and agent-relay access while denying management/Proxmox/DB access. Emulator verification precedes one matching board run with a live lease; no full HIL sweep. Existing `format`, `lint-go`, `test-go`, `ci-parity`, and all applicable registered gates remain required; warnings and skipped tests fail acceptance. No production deployment or legacy path removal occurs before these proofs and the operator decisions above.

Sources for this contract: [GitHub scale-set client v0.4.0](#), [pinned listener acknowledgment order](#), [pinned job message types](#), [GitHub workflow-jobs API](#), [GitHub check-run API](#), and [GitHub ephemeral runner guidance](#).

Implementation reference

This section closes the gap between the architectural intent and an executable engineering handoff. It is normative for the implementation lane unless it conflicts with `CLAUDE.md`, the machine-readable inventory, or a later owner decision. It does not authorize a live infrastructure or board mutation.

How to read this reference

Every component is assigned exactly one maturity label:

Label	Meaning
CURRENT	Present in the implementation branch and covered by at least focused tests. It is not necessarily production-ready.
TARGET	Approved behavior that still has to be implemented or wired.
BLOCKED	Implementation must fail closed until a named operator input, physical proof, or external credential exists.
REFERENCE	Existing repository behavior to preserve during migration; it is not automatically the final implementation.

The repository is the executable source of truth. This PDF supplies boundaries and contracts; these machine-readable files supply exhaustive item-level input:

- `tools/ra8ci-script-inventory.json`: every in-scope helper, its references, and verdict.
- `tools/ra8ci-work-units.json`: planner input and dependency information.
- `tools/ra8ci/catalog/tasks.json`: exact task declarations embedded in the current binary.
- `tools/ra8ci/migrations/*.sql`: exact database DDL and constraints.
- `tools/ra8ci/internal/protocol/protocol.go`: current agent wire structs and validation.

An engineer or AI must read those files before changing the corresponding subsystem. It must not infer a script mapping, database column, or protocol field from prose when a machine-readable definition exists.

As-built snapshot

This snapshot describes the `ci/ra8ci-implementation` code at `125f026ce4b596e7530c2743444f7ad07a78eaf9`, rebased onto `origin/dev` at `71feca26c2583545df7e06ee7c1e653e0a9d8025` on 23 September 2026. All 83 `ra8ci` commits replayed without conflict; the rebased feature branch was pushed. This document refresh is layered on that code snapshot. The branch includes the integrated `ci/orchestrator` work; the source branch is closed.

Item	State at this snapshot
Go surface	215 tracked Go files under <code>tools/ra8ci</code> , including tests and the server, agent, executor, store, board, scaler, GitHub, provisioning, and analytics packages.
Database	Forward-only PostgreSQL migrations <code>0001</code> through <code>0019</code> ; typed run, task, step, lease, runner, event, audit, HIL observation, and operation evidence exists in code. This is not evidence that a production database is deployed.
Task catalog	84 reviewed task definitions. 18 task definitions contain a native <code>ra8ci</code> : step; 66 remain transitional wrappers or external-command tasks. The native count is a direct catalog count, not a weighted completion percentage.

Item	State at this snapshot
Tracked helpers	532 .sh/.py files under scripts/ and 28 just/*.just files remain. The Phase 0 JSON inventory still reflects its earlier baseline and must be reconciled before planner batching.
Latest dev infrastructure baseline	Dev commits b7c0f7eb0 and 71feca26c harden disposable Linux/Windows Proxmox CI, Windows Server Core/WSL2 provisioning, log/status handling, and cleanup. The Linux full run was stopped before completion; Windows has no verified full run.
Completed checker ports in this lane	Five checker responsibilities now have native Go task implementations and have their old scripts removed in the same change: driver-assembly guard, goto/setjmp guard, GNU attribute guard, assert-casts, and C23 nullptr-only guard.
Server and agent	CLI, local executor, offline receipt spool/sync, authenticated HTTP server, PostgreSQL store, agent claim/log/result flow, deadlines, cancellation, and resource facts are implemented and tested.
Analytics	PostgreSQL timings and resource facts support report queries. Further report/API coverage and real production data are still required before scheduling tiers are tuned from evidence.
GitHub control	Scale-set client, durable inbox, policy, controller, and scaler state-machine packages exist. Critical gap: serve() does not compose the production controller, scaler handler, concrete guest bootstrapper, runner observer, or Terraform runtime.
Provisioning	Terraform control/runner definitions, a Terraform runner provisioner package, and the service Ansible role exist. The new dev lab path is a separate legacy baseline: just infra::lab::ci launches the detached server runner; just infra::lab::ci-terraform uses the local Terraform/Ansible driver. Production ra8ci Linux/Windows readiness and one-use JIT bootstrap are incomplete; no control-VM plan was applied.
Board and HIL	Durable lease/session/checkpoint/recovery primitives, API/client packages, neutral receipt primitives, and HIL timing policy exist. Production board-agent enrollment/device observation and bench.sh cutover are not complete.
Deployment	No control VM or production PostgreSQL has been created. Proxmox/GitHub credentials, approved storage/network values, and an off-VM backup destination remain operator inputs.
Acceptance	After the rebase, GOWORK=off go test ./..., go test -race ./..., go vet ./..., and static Linux-amd64/Windows-amd64 builds all exited 0. Logs and build artifacts are under /home/bsikar/ra8-verify/ra8ci-implementation/pdf-handoff-20260923/. The current dev full Proxmox CI runs are not verified: Linux has no final exit code and Windows has not completed. No live ra8ci GitHub dispatch, control-VM deployment, restore drill, Windows runtime, or hardware acceptance is claimed.

There is no defensible single project-completion percentage: one native checker port and a production dispatch controller are not comparable units. Use the status table below, the per-file JSON inventory, task catalog, and actual exit-code evidence instead. The earlier conversational estimate of roughly 85 percent remaining was deliberately approximate, not a release metric.

Implementation status and remaining work

The target architecture remains approved. The rows below separate code that exists from acceptance that is still required. SUBSTANTIAL means useful implementation exists, not production-ready; INCOMPLETE and NOT DEPLOYED are explicit non-completion states.

Workstream	Status	Remaining work and proof required
CLI and task execution	SUBSTANTIAL	Finish semantic task coverage, strict task arguments, output/artifact contracts, and parity for every migrated behavior. Keep safe local tasks usable offline; prove sync receipts are idempotent and never become CI attestations.
PostgreSQL and analytics	SUBSTANTIAL	Complete API/report surfaces and query tests for queue, step, resource, board, and runner timings. Prove append-only audit privileges, concurrency, retention, schema upgrade, and performance with representative disposable PostgreSQL data.
GitHub scale-set control	INCOMPLETE / FAIL-CLOSED	Compose the production GitHub session/controller in <code>serve()</code> ; add fixed reviewed config, singleton-controller fencing, durable reconciliation loops, concrete metadata resolution, guest bootstrap, runner observation, cancellation, and graceful shutdown. Until then the server must advertise zero managed capacity.
Disposable VM dispatch	INCOMPLETE	Resolve Terraform protection/identity/state contracts and implement Linux and Windows Ansible readiness. Deliver one-use JIT data through a protected channel only after reservation-bound guest proof. Test lost acknowledgments, restart, cancellation, busy drain, runner deregistration, and identity-checked cleanup on disposable VMs.
Board and HIL	INCOMPLETE / PRODUCTION-DISABLED	Implement and enroll the persistent board agent with key rotation, real device adapters, and independently verified neutral/restore proof. Integrate checkpoints, human > CI > AI ordering, yield/requeue, expiry, and recovery. Shadow then cut over from <code>bench.sh</code> without two live authorities; run emulator first and only one matching hardware case under lease.
Scripts and recipes	EARLY	Rebuild <code>tools/ra8ci-script-inventory.json</code> and <code>tools/ra8ci-work-units.json</code> from the current tree. Delete only evidenced dead files in planner-owned units. Port each used helper's behavior to a native task/package or explicitly retain it as provisioning/out-of-scope; repoint callers in the same change. 532 script files and 28 Justfiles remain as raw counts, not a promise to port every file.
Just, hooks, and workflows	EARLY	Repoint the remaining recipes and hook entries to semantic ra8ci task names. Keep GitHub's official Actions runner for workflow and <code>uses: semantics</code> ; switch <code>runs-on</code> demand to ra8ci-managed scale sets only after end-to-end capacity acceptance. Remove old dispatch entry points only after parity.

Workstream	Status	Remaining work and proof required
Control VM and operations	NOT DEPLOYED	Obtain approved least-privilege Proxmox and GitHub credentials, node/pool/template/storage/bridge/network inputs, and encrypted off-VM backup target. Plan the VM in the 9000+ range, review before apply, configure TLS/firewall/monitoring, and complete a restore drill before starting dispatch.
Security and deployment acceptance	INCOMPLETE	Test workflow admission, RCE boundaries, secret/JIT non-disclosure, Windows service ACLs, Linux process isolation, network denies, agent identity/fencing, database outage behavior, disk/backup/certificate alarms, recovery and rollback. Record operator approval; tests alone do not authorize live infrastructure changes.
Verification and integration	INCOMPLETE	Complete the required coverage threshold and all supported platform/runtime tests without skips. Run Terraform/Ansible validation, isolated Linux and Windows scale-set lifecycle acceptance, and the board emulator-then-single-hardware lane. The planner defines the MR stack; do not merge to <code>dev</code> without Brighton's approval.

Remaining-work ledger and completion evidence

These stable keys are for status discussions and implementation reports; they do not replace planner-owned issues or the machine-readable per-file work-unit list.

Key	Work unit	Done only when
R0	Inventory reconciliation	Current script, tool, infra, helper, Just, workflow, hook, Terraform, and Ansible rows are scanned and classified; absorbed files and caller links no longer point to deleted behavior; planner receives current JSON.
R1	Executor and evidence	Every required task has a reviewed schema, deadline, ordered steps, host facts, durable result/log evidence, offline behavior where allowed, and parity tests; no duplicate old implementation remains.
R2	Server production composition	A reviewed config builds exactly one fenced scale-set controller, handler, concrete provisioning/bootstrap dependencies, and reconcile/shutdown loops. Missing credential, bootstrapper, observer, audit, or database fails closed and advertises zero capacity.
R3	Linux/Windows guest lifecycle	First complete the legacy baseline below, then prove each ra8ci OS path binds VM identity to reservation and Ansible readiness before one-use JIT delivery. Each real isolated job drains, deregisters, and cleans up safely across cancellation, restart, and lost acknowledgments.
R4	Board agent and cutover	The board agent proves physical neutral state and recovery context; lease, yield, priority, and human wait semantics pass concurrency/failure tests. Emulator passes before a single matching leased hardware run, then old file/flock authority is retired.

Key	Work unit	Done only when
R5	Long-tail helper absorption	All used in-scope helpers are ported or have an explicit retained-owner rationale; dead items are deleted only from evidence. Caller changes and deletion land together, with no old/new overlap.
R6	Repository entry points	Just recipes, hooks, and workflow jobs invoke the same semantic task names. Third-party Actions remain on the official runner; ra8ci-managed labels are enabled only after R2/R3 acceptance.
R7	Control VM and recovery	Operator-approved infrastructure values exist; the persistent VM is applied only after review; PostgreSQL, encrypted off-VM backups, TLS, monitoring, restore, and rollback drills pass.
R8	Release verification	Required static/unit/race/coverage/database/platform/security/CI gates and non-skipped end-to-end acceptance pass with logs and exit codes. Production readiness and owner approval are recorded before external cutover.

Integration takeover handoff

This is the start-here section for the next engineer or AI taking over integration. It separates checked-in code from observed acceptance, gives the exact existing entry points, and names the prerequisites that must not be guessed. The handoff does not authorize a secret change, a Proxmox apply, a GitHub App change, a board operation, or a dev merge.

Repository and branch identity

Item	Known state
Repository	bsikar/ra8-firmware. Work only in /home/bsikar/worktrees/ra8ci-implementation on the dev box over ssh dev; never touch /Users/bsikar/Documents/github/ra8-firmware.
Feature branch	ci/ra8ci-implementation. The code snapshot before this doc refresh is 125f026ce4b596e7530c2743444f7ad07a78eaf9, pushed after rebasing.
Base	origin/dev at 71feca26c2583545df7e06ee7c1e653e0a9d8025; 83 ra8ci commits are replayed above it. dev is not this feature's merge target; Brighton controls the final integration.
Commit identity	Brighton Sikarskie <bsikar@tuta.io>. Keep commits free of AI attribution.
Build/test evidence	Post-rebase Go tests, race tests, vet, and Linux/Windows static builds exited 0. Current-runner acceptance is still incomplete; a Go package pass is not a Proxmox VM pass.
Output root	Put every build and test output under /home/bsikar/ra8-verify/ra8ci-implementation/<unique-unit>; do not use a shared or fixed /tmp build directory.
Hardware rule	The board is single-holder. For any later HIL change, run the matching emulator case first, then at most one matching hardware case under a lease. Do not run a full hardware sweep.

The detailed machine contracts remain in the repository: tools/ra8ci/catalog/tasks.json, tools/ra8ci/migrations/, tools/ra8ci/internal/protocol/protocol.go, tools/ra8ci-script-inventory.json, and tools/ra8ci-work-units.json. The inventory is stale relative to the current tree and must be regenerated before using it as an exhaustive deletion or migration plan.

What dev added and what it proves

The two relevant dev commits are b7c0f7eb0 (infra: harden disposable Proxmox CI runners) and 71feca26c (WIP: continue Proxmox Linux and Windows ephemeral CI). They change the existing disposable lab path: Linux guest setup and cleanup, a substantially expanded Windows Server Core/WSL2 playbook, Proxmox server-runner lifecycle and streamed logs, status/list/stop behavior, Windows-safe trusted Git/environment parsing, and a password variable named RA8_LAB_WINDOWS_PASSWORD.

The WIP commit reports that Linux reached the CI gate phase after fixing source-snapshot and rootless-container temporary-filesystem issues, but the run was intentionally stopped before completion. It has no final Linux exit status. Windows has not completed a verified full run. Infrastructure syntax/format checks passed, but the detached launch with the Windows password configured has not been verified. Do not translate any of these statements into an end-to-end pass.

The two lab commands are different implementations

Recipe	Actual path and what it validates
<code>just infra::lab::ci PROFILE</code>	<code>just/infra_lab.just -> scripts/dev/proxmox_lab_manage.py -> SSH upload of a source archive and runner -> detached sudo -n nohup launch of scripts/dev/proxmox_lab_server_runner.sh on Proxmox. The server runner uses direct qm lifecycle calls plus Ansible and applies the terraform tag. This is the detached server-runner baseline required first; it is not the local Terraform driver.</code>
<code>just infra::lab::ci-terraform PROFILE</code>	<code>just/infra_lab.just -> local scripts/dev/proxmox_lab_ci.sh; Terraform/Ansible lifecycle runs from the invoking control node. This is a separate implementation. A pass here does not validate the detached server-runner path, and neither command validates ra8ci production dispatch.</code>

For the requested baseline use `just infra::lab::check` first, then `just infra::lab::ci linux` to completion. Follow or reattach to output with `just infra::lab::logs linux` and query `just infra::lab::status`. The server log is `/var/log/ra8-lab/linux.log`; the Windows log is `/var/log/ra8-lab/windows.log`. Copy complete logs and terminal evidence to the unique verification directory on dev. The profile VM IDs are Linux 9000 from template 9001, and Windows 9010 from template 9011. Each guest should carry terraform, ra8-lab, and a unique run tag. Do not run both profiles concurrently.

Windows password is a hard gate

`scripts/dev/proxmox_lab_server_runner.sh` reads `RA8_LAB_WINDOWS_PASSWORD` and stops before Windows provisioning if it is empty. The current launcher in `scripts/dev/proxmox_lab_manage.py` constructs a remote command using `sudo -n nohup /bin/bash ...` but does not explicitly preserve or fetch this variable. No verified mechanism currently proves that a Proxmox-side environment value survives the SSH command, `sudo`, and the detached process. Ordinary `sudo` environment filtering means the current source is not proof that it will survive.

Before any Windows run, the owner/operator must establish a reviewed secret-delivery path on the Proxmox runner and verify presence through the full detached launch without printing the value. A narrowly-scoped root-owned service credential or protected environment file is preferable to broad environment preservation; if `sudo` preservation is proposed, allow only this named variable after reviewing the host policy. Do not put the password in Git, a command-line argument, shell history, Terraform state, Ansible inventory/artifacts, CI output, or logs. Never dump the environment or process arguments during verification; report only a boolean presence result and the run ID. If the safe path is not confirmed, stop before Windows provisioning and request operator setup rather than guessing.

Required disposable-run sequence and evidence

First run the read-only preflight `just infra::lab::check`. Then run Linux to completion with `just infra::lab::ci linux`; follow or reattach to output with `just infra::lab::logs linux`, and query `just infra::lab::status`. Preserve the complete log and the final runner exit code under a unique verification directory on dev. An interrupted/killed run is unknown, not pass or fail; do not report it as a completed measurement.

After Linux reaches a terminal result, query both `just infra::lab::status` and `just infra::lab::list`. Confirm VM 9000, its attached volumes, temporary bridge/firewall state, and its runner process are removed, unless `--keep` was deliberately requested for diagnosis. For a retained guest, record who requested preservation and destroy it with the approved destroy recipe after evidence collection. Confirm Terraform-created VMs carry the expected `terraform` tag before teardown.

Only after Linux cleanup and the password prerequisite are proven, run `just infra::lab::ci windows` to completion and follow with `just infra::lab::logs windows`, `just infra::lab::status`, and `just infra::lab::list`. Verify Server Core boots, WSL2 and the intended distro are usable, Ansible installs all declared tools including Go, the scratch `TMPDIR` resolves to the intended filesystem, logs remain visible through long tasks, and VM 9010 plus volumes/processes/network state are cleaned up. Save the full log and exact terminal exit code. For both profiles, test teardown separately for a completed run, a controlled provisioning/task failure, and a deliberate cancellation using `just infra::lab::stop PROFILE`. Do not cancel the required uninterrupted full run to test cleanup. If no safe failure-injection path exists, record that case as untested rather than improvising one. After each case, inspect `just infra::lab::status` and `just infra::lab::list`, then use approved read-only Proxmox inspection keyed to the exact run ID to verify VM, attached volumes, runner process, and temporary network/firewall state are gone. If a failed run remains active, record its run ID and status before requesting stop. If `--keep` was used, remove only that exact retained target with `just infra::lab::destroy TARGET` after evidence is saved.

For each profile record: commit under test, exact commands, start/end times, run ID, Proxmox VM ID and tags, log location, terminal exit code (or explicit unknown), gate failures grouped as infrastructure vs source/policy, cleanup checks, and any retained guest reason. A final `status` showing no active job is not by itself proof of no orphaned volume or process.

Failure triage and ra8ci integration order

Classification	Evidence and next action
Infrastructure failure	Missing package/tool, Ansible failure, guest boot/WSL2 issue, network, VM sizing, <code>TMPDIR</code> , absent/opaque logs, or leaked VM/volume/process. Fix the provisioning/runner path, rerun that profile from a clean state, and capture a new terminal result. A source-only finding may not mask an infrastructure failure.
Source or policy failure	Formatting, static analysis, repository policy, or firmware test findings after infrastructure gates execute correctly. Record the exact gate and output separately. The source issue may remain for this baseline handoff, but do not call the overall CI run green.
UNKNOWN	Signal, timeout outside a declared task deadline, or lost SSH/log session without server-side terminal evidence. Reattach through <code>logs</code> and <code>status</code> . If terminal result cannot be recovered, preserve evidence and rerun; interruption is never a pass.
Safety incident	Guest or cleanup identity differs from the recorded run/tags. Do not destroy by guessed VM ID or name. Stop dispatch, retain evidence, and use the exact run identity and approved operator recovery process.

Only after the legacy baseline has reproducible Linux and Windows results should the ra8ci lifecycle be integrated against it. Keep the handoff phases distinct:

1. Compose the production GitHub controller into `ra8ci server`: trusted fixed config, exactly one fenced controller, durable reconciliation, real guest bootstrap/observer, cancellation, and shutdown. Today `serve()` starts the authenticated API and maintenance reaper but does not start that production controller. The server must continue to advertise zero managed capacity until those dependencies and credentials are present.
2. Integrate the reservation-bound Terraform/Ansible lifecycle for Linux and Windows, including guest identity/readiness proof before one-use runner bootstrap. Preserve audit on retries, lost acknowledgments, cancellation, controller restart, busy drain, deregistration, and identity-checked cleanup.
3. Accept ra8ci dispatch on one isolated Linux job, then one isolated Windows job. Preserve GitHub Actions as scheduler and the official Actions runner as worker; ra8ci manages capacity and our task evidence, not GitHub's job protocol.
4. Once ra8ci parity is proven for a responsibility, repoint its `just` recipe/workflow and delete the superseded dispatch behavior in that same planner-defined change. Never operate old and new authorities for the same responsibility concurrently.
5. Only then expand script/task absorption, production deployment, board-agent enrollment, database restore drills, and fleet cutover. Board work still requires the emulator-first, one-matching-hardware-case rule.

The human handoff is intentionally scoped: this document gives the successor exact baseline paths and acceptance evidence. It does not grant access to secrets, the Proxmox API, a VM apply, GitHub App installation, physical hardware, or authority to merge to dev.

Full-stack ownership map

Layer	Owner	Contract
Developer UX	<code>just</code>	Stable recipes remain the memorable front door and invoke semantic <code>ra8ci</code> tasks.
Task semantics	<code>ra8ci</code> catalog and Go packages	Names, arguments, tiers, deadlines, host scope, steps, artifacts, retry class, and board policy.
Build graph	Zig	Compilation dependencies and parallel build graph. <code>ra8ci</code> schedules named work; it does not reproduce the Zig graph.
Workflow scheduling	GitHub Actions	Workflow/job graph, UI, checks, and the official runner job protocol.
Capacity control	<code>ra8ci server</code>	Scale-set demand, durable reservation, guest lifecycle, job correlation, and cleanup evidence.
Job execution	Official ephemeral GitHub runner	One job in one disposable Linux or Windows guest. <code>ra8ci</code> does not replace <code>Runner.Listener</code> or <code>Runner.Worker</code> .
Task execution	<code>ra8ci agent</code>	Outbound mTLS claim, exact source/catalog validation, process control, logs, facts, steps, and terminal receipt.
Guest readiness	Ansible	OS hardening, pinned official runner and agent install, services, and structured readiness proof.
Control persistence	PostgreSQL	Only the server has runtime database credentials; migration and operator roles remain separate.

Layer	Owner	Contract
Board execution	Persistent board agent	The sole software owner of J-Link, UART, reset, relay, and power operations after cutover.
Analytics	ra8ci server plus PostgreSQL	Typed timings, resource samples, queue phases, failures, cleanup, board waits, and reports.

The control VM contains ra8ci server and PostgreSQL. Repository-controlled job code never runs there or on the Proxmox host. Disposable guests have outbound access to the GitHub endpoints they need and the authenticated ra8ci relay, but no route or credential for PostgreSQL, the Proxmox management API, OpenBao, or board devices.

Command-line contract

Current commands

Command	State	Behavior
ra8ci tasks	CURRENT	Lists embedded task names.
ra8ci <task>	CURRENT	Runs only an embedded safe-local task after checkout/catalog validation. Creates an append-only local outbox record.
ra8ci sync	CURRENT	Uploads schema-v2 finished local evidence over mTLS. Legacy or unverified evidence never becomes a CI pass.
ra8ci server	CURRENT	Runs the authenticated control-plane HTTP server and expired-assignment reaper.
ra8ci agent	CURRENT	Claims one fenced task at a time from the server and streams logs/results.
ra8ci db migrate	CURRENT	Applies forward-only migrations using the separate migration DSN.
ra8ci report slow	CURRENT	Bounded server-side median/p95/max plus clearly unit-labeled Linux load-per-core or Windows CPU-busy context and RAM use; host context is not task-attributed CPU.
ra8ci run submit --idempotency-key KEY TASK...	CURRENT	Submits up to 100 catalog-matched read-only tasks from a clean pinned source snapshot. Retries with the same key return the same run.
ra8ci run status <id>	CURRENT	Fetches an authorized run status over mTLS and verifies the returned run ID, cancellation request, and actor.
ra8ci run cancel <id>	CURRENT	Idempotently records a durable, audited cooperative cancellation request. Queued tasks cancel immediately; assigned tasks stop at the next fenced heartbeat and must return terminal evidence.
ra8ci run logs <run> <attempt>	CURRENT	Pages through currently available attempt logs over mTLS, checking sequence and SHA-256 before writing stdout/stderr.
ra8ci board status <id>	CURRENT	Read-only snapshot through the authenticated board API.
ra8ci board take <id>	CURRENT	Queues a human-priority request with reason and duration, prints request and lease IDs before waiting, and cancels the waiter on interrupted wait where safe.

Command	State	Behavior
ra8ci board cancel <id> <request> <lease>	CURRENT	Withdraws only a still-queued waiter using its board/request/lease IDs and authenticated owner identity. If the lease was already granted, cancellation refuses and does not release hardware.
ra8ci board extend <id> --why WHY --duration DURATION	CURRENT	Extends the current user's lease using a private, owner-only token saved after grant; server-side class ceilings and contention rules remain authoritative.
Board release/checkpoint CLI	BLOCKED	Release still requires verifier-backed neutral evidence from an enrolled/fenced board agent; checkpoint requires an implemented agent operation and an approved fixture profile.

Current process exit meanings are 0 for command/task success, the child exit for a completed local task, 124 for a task deadline, 130 for cancellation, 2 for CLI usage, and 1 for control-plane or internal failure. The target CLI must replace the generic 1 with stable documented categories without changing an underlying task's meaningful child result.

Target commands

Add only after their server contracts and tests exist:

- ra8ci run retry|explain <run-id>.
- ra8ci board checkpoint|release|queue|doctor.
- ra8ci board flash|probe|reset|serial|debug through allowlisted board-agent operations; never raw remote shell text.
- ra8ci report critical-path|resource-fit|board.
- --json for stable machine output and --local or --dispatch only where both routes have identical task semantics.

HTTP and agent protocol contract

Implemented HTTP routes

Verb	Path	Purpose
GET	/health/live	Process liveness only.
GET	/health/ready	Database, audit write, and schema readiness.
POST	/v1/runs	Admit a catalog-matched run with source identity and idempotency key.
GET	/v1/runs/{id}	Authorized run state.
GET	/v1/runs/{id}/events	Authorized append-only event stream with bounded contiguous cursor pages.
GET	/v1/runs/{id}/logs	Authorized attempt-scoped, bounded, digest-checked log pages.
POST	/v1/local-runs/sync	Durable local evidence ingest, separate from trusted CI runs.
GET	/v1/reports/slow	Bounded repository-scoped performance report.
POST	/v1/agents/me/claim	Authenticated fenced task claim.

Verb	Path	Purpose
POST	/v1/assignments/{id}/ack	Exact assignment/source/catalog acknowledgment.
POST	/v1/attempts/{id}/logs	Gap-free, digest-checked log chunks.
POST	/v1/attempts/{id}/result	Terminal receipt and step evidence.
POST	/v1/agents/me/heartbeat	Host facts, phase, cancel, and yield intent.

Board routes currently include status, take, waiter cancel, checkpoint, release, extend, neutral challenge, agent acknowledgment/observation/unavailable, recovery start, and quarantine under `/v1/boards/{board-id}`. A route existing does not make it production-enabled: absence of a neutral verifier, board-agent identity, or approved fixture profile must return unavailable or denied and must be audited.

Required API completion

Before stable v1 is declared, generate an OpenAPI document from the actual structs and add contract tests for it. Implement log streaming, enrollment, artifact upload/download, resource batches, operator recovery, drain, and the remaining reports. Run events are implemented with bounded pages, repository authorization, contiguous sequence validation, and JSONL CLI output. Every mutation must define maximum body size, strict unknown-field rejection, authenticated role, idempotency scope, correlation ID, retryability, and reconciliation after an ambiguous response.

HTTP status policy is: malformed 400, unauthenticated 401 at the TLS/API boundary, authenticated-but-forbidden 403 where disclosure is safe, concealed foreign resources 404, stale/conflict 409, expired authority 410, too large 413, unsupported media 415, rate limit 429, and unavailable dependency 503. Error bodies use versioned `application/problem+json`; internal SQL, paths, credentials, and foreign resource existence are never exposed.

Agent protocol invariants

Protocol version 1 carries host facts, a task reference, exact catalog digest, recursively verified source identity, assignment ID/version, fencing token, and a relative remaining deadline. Server wall-clock deadlines are never trusted as a way to gain time on another host. The agent validates its local clean snapshot immediately before execution, seeds a local timeout, acknowledges exact evidence, uploads ordered digest-checked chunks, and reports one terminal receipt. A complete success requires every declared step, successful step exits, final sequence, byte counts, and stream digests. A stale fence, missing log, mismatched source, expired certificate, or incomplete step makes the attempt non-successful.

Remote script dispatch remains closed unless `RA8CI_AGENT_TRUSTED_COMMIT` is an exact reviewed commit. This is transitional. The final design executes absorbed task implementations in disposable guests and does not give pull-request code access to the agent's mTLS private key or management credentials.

Identity and authorization matrix

Principal	Credential	Maximum authority
Observer	Short-lived client certificate	Read authorized runs, reports, and board status.
Submitter	Short-lived client certificate	Observer plus admit reviewed non-board tasks for an authorized repository.
Human board user	Short-lived client certificate	Request human-priority lease and use allowlisted operations while holding the exact active generation.

Principal	Credential	Maximum authority
CI/Al actor	Server-correlated job or agent certificate	Only its configured class; it cannot claim human priority.
Execution agent	Reservation-bound mTLS certificate	Claim catalog tasks for its host class and return evidence. No database or Proxmox credential.
Board agent	Pinned host identity plus receipt signing key	Acknowledge generations and perform approved physical operations.
Provisioner	Dedicated control-VM identity	Only approved Terraform/Ansible lifecycle for reserved pools, templates, storage, and bridges.
Operator	Separate interactive identity	Migrations, fixture approval, recovery, exceptional cleanup, and audited credential rotation.

Certificate fingerprints map to `api_principals`; repository-scoped roles live in `api_grants`. The runtime database role may insert audit/event rows but cannot update or delete append-only history, alter identity/grants/fixture profiles/migrations, or execute DDL. Migration and operator credentials never enter the server environment used for normal requests.

PostgreSQL schema and state machines

Migration ledger

Version	Migration	Responsibility
1	initial	Runs, task graph, attempts, steps, resource samples, artifacts/logs, boards, GitHub inbox/jobs, lifecycle/external operations, idempotency, events, audit, and API identity.
2	board_state	Serialized board snapshot and append-only board events.
3	board_neutral_challenge	Approved fixture profiles, recovery context, and one-use neutral challenges.
4	agent_dispatch	Durable assignment, fencing, log receipt, and attempt lifecycle support.
5	offline_sync	Separate append-only local runs and local step evidence.
6	runner_vms	Immutable runner reservation and external operation intents.
7	runner_vm_cleanup_fence	Monotonic cleanup-requested fence so replay cannot restart a completed guest.
8	runner_vm_exact_digests	Exact template/live config digests, permanent scale-set/job uniqueness, runner safety identity, and approved reconciliation.

Migrations are forward-only, sequential, embedded in the binary, protected by a PostgreSQL advisory lock, and recorded in `schema_migrations`. Runtime startup rejects an older or newer schema and rejects a role with forbidden privileges. Never edit an applied migration; add a new numbered migration.

Core lifecycle rules

- Run: `queued` -> `running` -> `terminal`; a terminal run has an end time and explicit execution, cleanup, and evidence outcomes.
- Task: `scheduled` -> `running` -> `succeeded`|`failed`|`timed_out`|`cancelled`|`preempted`|`lost`|`skipped`.
- Assignment: `issued`, `acknowledged`, `running`, then one terminal state. Expired assignments are fenced before retry and cannot leave a run permanently open.

- Board: ready, grant pending, active, yield requested or draining, recovery required, recovering, or quarantined. Exactly one pending/active authority per board.
- Runner VM: reserved, provisioning, ready/registered, draining, stopped, destroyed, or failed/unknown according to the DDL. Cleanup request is monotonic. An unresolved external operation prevents blind replay.
- Audit, `run_events`, `board_events`, local evidence, and operation history are append-only.

Every state transition and external side effect uses intent-before-action: commit the operation identity first, invoke the external system second, then record observed evidence. After connection loss, reconcile by immutable provider identity; never repeat clone, start, flash, stop, destroy, or check publication merely because the client did not receive a response.

Task catalog and script migration

Transitional catalog

Task	Tier	Scope	Deadline	Current step
format	required	local write	900 s	bash scripts/checks/format_tree.sh
format-check	required	local read	900 s	bash scripts/checks/format_tree.sh --check
lint-go	required	local read	1200 s	just quality::local::gate lint-go
test-go	required	local read	1800 s	just quality::local::gate test-go

These are executor-stage adapters, not finished ports. A completed task implementation no longer shells out to the helper whose behavior it replaces.

Catalog schema

Each task declaration has: stable name and version; tier `required`, `optional`, or `nightly`; scope; OS allowlist; capability list; strict arguments schema; total deadline; board policy; ordered typed steps; declared artifacts; bounded retry policy; and resource hints. Board tasks additionally declare cancellation class, maximum safe segment, restore/probe bound, retry-from-baseline rule, fixture profile, and checkpoint positions. Unknown fields, executable paths, environment additions, or arguments fail catalog validation.

Exhaustive migration rule

The Phase 0 inventory captured 588 helper rows at its historical `ci/orchestrator` snapshot: 31 dead and 557 used. It is no longer an exact picture of the implementation branch: dead files were removed and five used checker responsibilities were absorbed. Recompute the inventory before further planner batching. A used file is not necessarily a public task: entry points become catalog tasks; shared parsing/model code becomes internal Go packages; Terraform and Ansible remain the provisioning layer; substantial firmware/deliverable programs remain out of scope.

For every used file, the migration ledger must record:

`old_path`, all callers, responsibility, new task or package, task version, host/OS, old success/failure/timeout/artifact/selftest fixtures, new parity proof, caller changes, deletion change, rollback boundary, and state.

The safe order for each responsibility is: capture golden behavior; implement the Go replacement; run both against non-hardware fixtures; repoint every caller; delete the old helper in the same change; run the registered gate; and never enable old and new implementations concurrently. HIL parity runs emulator first and exactly one matching hardware case under a lease.

GitHub Actions integration

Approved model

GitHub remains workflow scheduler and check UI. The official ephemeral runner executes `run:`, JavaScript, composite, container, and third-party uses: actions. `ra8ci` uses the pinned `github.com/actions/scaleset v0.4.0` client to listen for scale-set demand, persist messages before acknowledgment, apply trusted admission policy, reserve capacity, and reconcile guests. It does not implement the private Actions worker protocol.

Required GitHub App configuration

TARGET: accept explicit App ID, installation ID, private-key file, repository identity, runner scale-set ID, maximum capacity, approved workflow paths and refs, events, job names, and labels. Use `https://api.github.com` and validate dynamically returned Actions service/queue HTTPS hosts against GitHub's documented domains. Disable proxy inheritance for management credentials. Reject GHES or alternate API origins unless a separate reviewed configuration names them.

The App receives only permissions needed to read workflow jobs, manage the repository runner scale set, and publish the dedicated lifecycle check. Private key and tokens live only on the control VM. The official SDK may renew its admin/queue tokens; `ra8ci` still bounds session lifetime, validates returned scale-set identity and URL, and stops advertising capacity on authentication or inbox failure.

Durable message and job policy

Persist scale-set ID, session ID, message ID, and normalized nonsecret payload before acknowledging the message. A divergent replay is a conflict. Admission compares repository ID, workflow path and pinned ref, event class, job name, and labels against server-owned policy. Fork PRs and untrusted refs cannot obtain board or provisioner authority. Job text, labels, environment variables, and repository files never become Terraform, Ansible, Proxmox, or host shell arguments.

Runner teardown is permitted only from the durably persisted terminal `JobCompleted` event whose job/request/run/repository/ref and runner ID/name exactly match the reservation, and which contains a nonempty result and finish time. That GitHub terminal event is the no-active-job proof; independently query the configured scale-set administration API, verify the exact runner identity, remove it, then query again and require absence. An already-absent exact runner is an idempotent success. API uncertainty, a mismatched runner, missing completion facts, or continued registration fails closed. VM stop and Terraform cleanup remain separately fenced by fresh drain evidence, distinct cleanup approval, stopped-state identity, and exact preserved Terraform state.

TARGET: publish one separate `ra8ci` lifecycle check per GitHub run/attempt. It remains pending through guest creation, runner registration, execution, evidence, deregistration, and cleanup. Native job success cannot override failed or incomplete lifecycle evidence. An ambiguous Checks API write is reconciled by SHA, check name, App ID, and external ID.

JIT and guest correlation

Issue one-use JIT configuration only after the declared guest identity and Ansible readiness proof are verified. Bind it to reservation, scale set, OS, job request, and expiration. The guest agent accepts a task only after server-side correlation of repository, exact head SHA, workflow policy, and official runner identity. Completion revokes the grant and drains the agent. A compromised disposable guest can falsify guest-origin measurements, so those measurements remain labeled; it cannot gain management credentials or authority for another job.

Terraform and Ansible runner provisioning

Mandatory production path

Production guest mutation is Terraform -> Ansible -> official runner/agent, behind the scaler's injected Provisioner interface. The direct Proxmox Go client is not the production mutation backend. It may provide pinned-CA read-only identity and task reconciliation, and its fake-backed lifecycle tests exercise ambiguity handling. Enabling its clone/start/stop/destroy methods in production requires a separate explicit architecture decision and safety review.

Discovered incompatibilities to resolve

Conflict	Required resolution
Destroy protection	Existing lab VM modules use <code>protection=true</code> and <code>lifecycle.prevent_destroy=true</code> ; the old script bypasses Terraform with <code>direct qm destroy</code> and <code>terraform state rm</code> . Do not reproduce that bypass. Design a reviewed disposable-resource module and tested cleanup/import contract.
Identity marker	Existing description uses <code>RA8_LAB_RUN=<16hex></code> while the new lifecycle expects reservation and operation UUIDs. Define one immutable marker schema emitted by Terraform and verified everywhere.
Names	Existing Windows names require <code>ra8-lab-win-*</code> ; the prototype scaler assumed <code>ra8-lab-ci-<vmid></code> . Define OS-specific naming in one provider-owned function.
VMID and bridges	Current disposable lab reserves 9000-9099 and exact <code>vmbr8/vmbr9</code> paths. Persistent control VM is 9100 or greater and must use an independently approved management bridge.
Operation ledger	Runner VM operations currently name Proxmox UPID evidence. Generalize through a new forward migration to provider kind, operation identity, plan digest, state identity, and reconciliation proof before Terraform is the production backend.

Provisioner contract

Inputs are typed values, never a command string: reservation/job identity, OS profile, template ID and digest, node, pool, storage, bridge, VMID/name, immutable marker, source version manifest, and state root. Before invocation, validate every value against an operator-approved allowlist. Use fixed argv, an empty/minimal environment, pinned Terraform/provider versions, `TF_DATA_DIR` and state under a private per-reservation directory, and no repository-controlled extra variables.

Persist operation intent and expected identity; run `terraform init -backend=false, validate`, and a saved plan; hash plan, lock file, module tree, variables-without-secrets, and provider versions; then apply that saved plan. Preserve state and logs after any uncertainty. Reconcile actual identity and state before retry. Cleanup requires completed/failed job intent, independently verified runner deregistration and idle state, exact marker/template/pool/storage/bridge/non-template identity, a saved cleanup plan, and explicit outcome. Foreign or ambiguous resources are left intact and raised for operator recovery.

Ansible readiness contract

Create `infra/ansible/playbooks/ra8ci-lab-linux.yml` and `ra8ci-lab-windows.yml`. They reuse reviewed connectivity/hardening roles but install pinned, digest-verified official runner and ra8ci binaries as narrow service identities. They accept an exact generated inventory, version manifest, one-use enrollment token, and later JIT config through a protected secret channel. They return structured JSON proof containing reservation marker, OS/arch, binary digests, service accounts, time/skew, disk, network

tests, agent enrollment, and runner readiness. A nonzero, warning, changed-when-forbidden, skipped, malformed, or identity-mismatched result blocks JIT and cleanup.

Control VM deployment

CURRENT: `infra/terraform/environments/ra8ci-service` declares a separate persistent VM with an explicit enable gate, VMID at least 9100, reviewed template, dedicated pool, boot/data storage, private management bridge, static network, and separate data disk. The first plan remains stopped unless `start_after_review` is explicitly set. `infra/ansible/ra8ci-service` declares PostgreSQL, separate migration/runtime/operator roles, local socket access, TLS server service, and encrypted pgBackRest scheduling.

BLOCKED: no apply may occur until all inputs in the operator registry below are supplied and reviewed. A successful static Terraform validate is not permission to create a VM. No personal Tailscale discovery, Mac keychain credential, guessed Proxmox node, default bridge, or placeholder backup target may be used.

Deployment order is:

1. Record approved inputs and credential owners without secrets in Git.
2. Run Terraform `init/fmt/validate` and produce a saved plan from the control environment.
3. Review exact VM identity, network, disks, pool, and no repository job execution on the host/control VM.
4. Apply with a dedicated least-privilege Proxmox/OpenBao identity; capture plan and task evidence.
5. Run Ansible in check mode, review changes, then configure PostgreSQL/backups and the unprivileged service.
6. Apply migrations with migration role; start server with runtime role and mTLS.
7. Prove live/ready separation, authorization denial audit, backup freshness, and restore into an isolated VM.
8. Enroll no runner or board agent until its own acceptance gate is complete.

Board agent, leases, and HIL

Authority and physical boundary

The persistent board agent is the only software allowed to touch J-Link, UART, reset, relay, or board power after cutover. Disposable guests, hooks, workflows, and legacy scripts lose direct device/SSH authority. Physical/root access remains outside software fencing and must be an audited operator procedure.

The lease service provides human > CI > AI priority, FIFO within class, one holder, bounded duration, cooperative yield, and crash durability. Priority requests never yank an indivisible operation. An active holder checks yield at declared checkpoints, neutralizes, releases with signed proof, and requeues if work remains. Expiry forbids starting another segment but does not claim that an already-running electrical operation became safe.

Signed neutral proof

CURRENT: the neutral receipt format is canonical JSON with domain separation and an allowlisted Ed25519 key ID. It binds challenge ID/nonce, board, lease, generation, snapshot version, agent high-water mark, approved fixture revision/profile/restore policy, recovery plan, issue/expiry, observation time, and fixture evidence digest. The store consumes each challenge once inside the board transition transaction.

BLOCKED: production needs a privileged `NeutralObserver` adapter that actually proves process/fd quiescence, no unexpected device owner, correct probe/board serial, approved reset/relay/power state, VTref/sensors where available, and baseline firmware health. No default observer, shell fallback, or connectivity-only probe is acceptable. Key generation, storage, rotation, revocation, and board-agent enrollment must be operator-approved.

HIL specification and timing

CURRENT: the literal-only Go HIL parser accepts the current allowlisted `HIL_*` assignments without sourcing shell or evaluating interpolation, and its tests parse all 208 current `examples/**/hil.conf` files. It separates observation timeout from flash/restore safety bound.

The timeout decision filters exact comparable successful, evidence-complete, uncensored history. With sufficient samples it uses the conservative historical estimator; with sparse data it uses declared `HIL_TIMEOUT_S`, otherwise 30 seconds. An optional approved maximum clamps learning. Flash/restore bound and recovery margin are independent mandatory inputs for board dispatch. A human yield ETA uses request-to-neutral history, not the firmware observation timeout. Unknown bounds reject automatic board dispatch.

The first live adapter must support allowlisted flash, probe, reset, serial, and debug lifecycle operations with fixed arguments and supervised process groups; record immutable image digest and full arguments; verify unstripped firmware before transformation; restore the recorded baseline after preemption; and quarantine on identity, clock, neutral, recovery, or high-water disagreement.

RCE and trust-boundary threat model

Threat	Boundary	Required control
Untrusted PR command	GitHub message/workflow to control VM	Server-owned policy maps verified meta-data to task IDs. Never pass workflow strings to shell, Terraform, Ansible, or Proxmox.
Agent key theft	Job process to ra8ci agent	Different OS identities and ACLs; job accesses only a narrow local task relay. Long-term agent key is unreadable. Disposable identity expires after one job.
Workspace swap	Runner checkout to agent	Canonical workspace under dedicated root, no symlink escape, clean recursive snapshot, exact verified head SHA immediately before task.
Credential exfiltration	Guest to management plane	No DB/Proxmox/OpenBao credentials in guest; management firewall denies routes; egress allowlist; redact logs and audit access, not values.
Replay after crash	Server to external systems	Intent-before-action, permanent idempotency identity, fencing generation, explicit reconciliation, no blind retries.
Foreign VM deletion	Provisioner cleanup	Exact marker/name/VMID/node/pool/storage/bridge/template/config digest and independent stopped/idle/deregistered proof. Ambiguity preserves the VM.
Board takeover	Client/server/board agent	mTLS role, class fixed by server, pending grant until agent high-water acknowledgment, local monotonic deadline, signed neutral release.
Database privilege escalation	Runtime process to PostgreSQL	Separate roles; no DDL; no identity/config mutation; no update/delete of append-only history; startup privilege audit.

The security review must additionally cover denial of service, certificate clock skew, log-volume caps, JSON depth/number handling, archive traversal, artifact digest confusion, submodule replacement,

Windows service ACLs, Linux cgroups/namespaces, proxy environment inheritance, DNS rebinding, and dependency provenance. A passing unit test is not proof of host isolation; acceptance includes firewall and service-account tests in disposable guests.

Configuration and operator input registry

Application environment

Current configuration names are: RA8CI_DATABASE_URL, RA8CI_MIGRATION_DATABASE_URL, RA8CI_LISTEN_ADDR, RA8CI_TLS_CERT, RA8CI_TLS_KEY, RA8CI_CLIENT_CA, RA8CI_SERVER_URL, RA8CI_SERVER_CA, RA8CI_CLIENT_CERT, RA8CI_CLIENT_KEY, RA8CI_AGENT_CERT, RA8CI_AGENT_KEY, RA8CI_AGENT_ROOT, RA8CI_AGENT_POLL_WAIT, RA8CI_AGENT_TRUSTED_COMMIT, RA8CI_REPOSITORY, and RA8CI_STATE_DIR. Proxmox test/prototype code also recognizes RA8CI_PROXMOX_API_TOKEN, RA8CI_RESERVATION, and RA8CI_OPERATION; these do not authorize production direct mutation.

Secrets are read from protected files or a reviewed secret provider, not committed environment templates, command arguments, Terraform state, logs, or audit metadata. Service startup validates file ownership/mode and rejects symlinks where a credential path is expected.

Required operator inputs

Domain	Required values	Absent behavior
Control VM	Proxmox HTTPS endpoint/CA, node, VMID $\geq$ 9100, template ID/digest, name, pool, boot/data storage, management bridge, IP/gateway/DNS, admin keys.	No Terraform apply.
Credentials	Least-privilege Proxmox API token or approved OpenBao AppRole path, owner, rotation, and revocation.	No provisioner.
Backups	Encrypted off-VM target, credentials, retention, RPO/RTO, restore destination, and drill owner.	Server readiness fails for production.
PKI	CA owners, server/client profiles, principal roles, lifetimes, renewal, revocation, and emergency rotation.	Only local offline tasks work.
GitHub	App ID, installation ID, private key, repository ID, scale-set IDs/labels, workflow/ref/event/job policy, lifecycle check name.	Advertise zero capacity.
Disposable guests	Linux/Windows template IDs/digests, VMID/name pools, node/pool/storage/bridge, runner/agent versions/digests, firewall policy.	No JIT and no guest creation.
Board	Board ID/serials, signing key, fixture revision/profile, baseline digest, device topology, neutral predicates, recovery plans, physical-clear owner.	Mutating board commands unavailable.

These are deployment data, not architecture ambiguity. The implementation must expose a validation report listing each absent item without printing secret values.

Operations and recovery runbook contract

Normal operation

The server starts only after schema and privilege checks. `/live` means process alive; `/ready` means database/audit writable, expected schema, scheduler/reaper healthy, disk below critical threshold,

and production backup freshness when production mode is enabled. Drain stops new run assignments, capacity reservations, and board grants while allowing bounded safe work and cleanup to finish.

Metrics and alerts cover run/task/step duration, queue wait, provision/registration/cleanup phases, CPU/RAM/load/I/O with provenance, log lag, dropped samples, agent heartbeat, database connections/WAL/backup age, disk 80/90 percent thresholds, certificate 30-day expiry, stuck cleanup, orphan guests, board recovery, yield overruns, and quarantines. Metrics never contain secrets, source text, or private board data.

Backup and restore

Back up PostgreSQL with encrypted pgBackRest full/differential/WAL retention to the approved off-VM destination. Back up only immutable artifact objects and manifests required to resolve database references; never claim a database restore is complete if artifacts or Terraform state are missing. Store per-run Terraform state in protected durable storage with its plan/module/provider/identity proof.

A restore drill creates an isolated control VM/database, verifies checksums and point-in-time recovery, runs migrations only if explicitly part of the drill, starts a non-provisioning server, reconciles agent/runner/board high-water state, and proves no live lease or external operation is silently resumed. Record timestamps, RPO, RTO, missing objects, and operator sign-off.

Upgrade and rollback

Drain, back up, verify restore, deploy the new binary, apply forward migration with the migration role, run readiness and protocol compatibility tests, then resume. Database migrations have no automatic down path. Binary rollback is allowed only when it accepts the current schema; otherwise restore to a new isolated database and reconcile. Agents outside the supported protocol window are drained, not assigned undefined work.

Incident rules

- Database unavailable: safe local tasks run and remain unsynced; dispatch and board mutation fail closed.
- GitHub unavailable: advertise zero new capacity; retain durable inbox/reservations and reconcile before resuming.
- Proxmox/Terraform uncertainty: preserve state and guest; do not replay or delete until exact identity is observed.
- Agent lost: fence assignment, preserve evidence, retry only according to catalog and idempotency policy.
- Board agent lost or server restored behind agent high-water: recovery required or quarantine; never grant next holder.
- Backup stale, disk critical, certificate expired, or audit unwritable: readiness fails and new external work stops.

Implementation sequence and definitions of done

The planner still decides issue and MR boundaries. The engineering dependency order is:

1. **Baseline closure.** Commit current code on its dedicated branch only after complete race, integration, coverage, format, static analysis, schema-from-empty, Linux build, Windows cross-build, and PDF checks. Done means no warning, skip, or below-floor coverage is reported as pass.
2. **API and schema stabilization.** Generate OpenAPI, finish events/cancel/enrollment/artifacts/resources, generalize provider operation evidence, and add migration/privilege/concurrency tests. Done means protocol and DDL are versioned and crash-replay tested.
3. **Control VM readiness.** Obtain inputs, plan/apply the persistent VM, configure PostgreSQL/backups/PKI, and pass an isolated restore drill. Done means no repository job can execute there and production readiness observes backup freshness.

4. **GitHub bootstrap.** Configure the App and official scale-set session, durable inbox, admission, JIT delivery, and lifecycle check. Done means lost acknowledgments and controller restart do not lose or duplicate a job.
5. **Terraform/Ansible disposable runners.** Resolve module protection/marker/name issues, implement provider adapter and readiness playbooks, then prove one Linux and one Windows ephemeral job, cancellation, crash reconciliation, deregistration, and identity-checked cleanup.
6. **Board agent.** Implement device adapters and neutral observer, seed approved fixture/recovery profiles, enroll keys, and pass simulator/emulator then one matching leased hardware case. Done means no lower-priority work starts while a human waits and no release occurs without neutral proof.
7. **Absorb helpers.** Follow the planner's units: dead deletion first, then one responsibility at a time with parity/caller repoint/deletion. Done means the old helper is gone and the semantic task passes its registered gate.
8. **Entry-point cutover.** Repoint just, hooks, and workflows after each task exists. Done means one responsibility has one active implementation and the same task semantics run locally and in CI.
9. **Production acceptance.** Execute security, restore, firewall, performance, orphan cleanup, board recovery, and operator runbooks; record evidence and owner sign-off before making checks required or removing fallback fleet paths.

Executable acceptance matrix

Gate	Environment	Pass condition
Go format/vet/static-check	dev box	All packages; warnings are failures; no changed generated catalog digest.
Race unit suite	Linux dev box	go test -race -count=1 ./... exits 0 with no skip used as success.
PostgreSQL integration	Disposable loopback test DB	Fresh schema 1-8 plus concurrent store/server tests exit 0; production DSN is refused.
Coverage	Linux dev box	One merged profile across packages and integration tests is at least 85 percent; the gate must not take the maximum of per-package module-wide percentages.
Static binaries	Linux build host	Linux amd64, Linux arm64, and Windows amd64 cross-builds succeed without in-tree caches; Windows process behavior also receives a real Windows runtime test.
Terraform/Ansible static	dev box	Pinned init without backend, fmt, validate, lint, syntax, and negative approval gates pass.
Control restore	Isolated VM	PostgreSQL, artifacts, and state restore meet approved RPO/RTO and resume no stale authority.
GitHub prototype	Isolated scale set	One Linux and one Windows job, representative run: and uses:, JIT expiry, cancellation, restart, same-SHA rerun, failure, and cleanup.
Network/RCE	Disposable guests	GitHub and ra8ci relay reachable; Proxmox, database, secret store, management network, and another job's identity unreachable.
Board	Simulator, emulator, then one board	Priority/yield/requeue, timeout, signed neutral release, baseline restore, agent loss, high-water rollback, and quarantine pass.

Gate	Environment	Pass condition
Repository gates	dev box and CI	All applicable registered gates run; warning, skip, timeout, or incomplete evidence is not green.

Build and test output goes under the per-worktree verification root, never in-tree and never a shared /tmp build directory. Every remote build is detached to a log with a separately captured exit code, and the log is read before recording a pass.

Handoff checklist for an engineer or AI

Before writing code, confirm the isolated worktree and base, read CLAUDE.md, inspect the accepted inventory/work units, and identify the planner-owned unit. Do not inspect or modify Brighton’s active checkout. State the exact files owned, preserve concurrent edits, and avoid live external mutations unless the unit explicitly includes an approved acceptance operation.

Before declaring a unit complete, provide: branch and head SHA; one commit for that MR unit; changed/deleted paths; old behavior and exact new owner; migration/API/catalog version changes; security assumptions; detached verification commands/logs/exit codes; skipped or unavailable environments; coverage; and remaining operator inputs. Do not merge to dev, create issues, invent the MR stack, bypass CI, or call an untested fallback a pass.

Decision register and discussion prompts

Confirmed decisions: Safe local tasks run offline when the server or PostgreSQL is unavailable. Their receipts remain visibly unsynced until uploaded; board and dispatched tasks fail closed. Thirty seconds is the default human handoff target, not a hard takeover guarantee: an indivisible flash or recovery can overrun it visibly, and no next lease is granted until the board is verified safe. A hung or unverifiable board is quarantined.

Confirmed design direction: Go; one product/three modes; protected persistent control VM on Proxmox with colocated PostgreSQL; agents connect out; ra8ci owns lifecycle state while Terraform/Ansible provision; Zig owns the build graph; semantic tasks behind just; cooperative board priority human > CI > AI; detailed per-step and per-resource measurements. For GitHub Actions, ra8ci controls an ephemeral runner scale set while the official runner executes jobs in disposable guests; it does not replace the worker protocol.

Confirmed decisions, 23 September 2026 revision: workflow_job webhooks plus just-in-time runner registration are the primary demand signal, with the scale-set client retained as a feature-gated second adapter behind one internal interface. An unclaimed-runner deadline with a reconciliation pass is a required component of the first dispatch release. The Windows lab password moves out of the environment into a root-owned file read at the point of use. Self-hosting a forge is rejected as an answer to this problem. ra8ci publishing its own check runs is the intended end state and the second migration, sequenced after dispatch is proven.

Required operator inputs before production:

- What approved off-VM destination holds encrypted PostgreSQL and artifact backups, and which dedicated Proxmox API/GitHub App credentials can be provisioned? No deployment proceeds by guessing either.
- The first useful release is CLI/API plus streamed logs; a web console is outside initial acceptance and may follow measured usage.
- Who may approve keep-guest and manual recovery actions? Recommendation: a narrow operator role, never ordinary CI or AI.
- Existing third-party uses: steps remain supported by the official runner. Project-specific checks migrate to named ra8ci tasks without rewriting unrelated Actions behavior.

Evidence and sources

Repository sources inspected read-only: `infra/fleet.yml`; `infra/terraform/README.md` and lab modules; `infra/ansible/playbooks/proxmox-lab-linux.yml` and `proxmox-lab-windows.yml`; `scripts/dev/proxmox_lab_ci.sh` and `proxmox_lab_server_runner.sh` on published dev at `a7666d637`. HIL timing and recovery evidence on that dev tip: `scripts/hil/all.sh`:21-32, 375-462, 487-531, 543-596; `scripts/hil/lib/hil_conf.sh`:97-109; `examples/ek_ra8d2/hw_validated/hil/usb_selftest_soak/hil.conf`:9 (200-second observation); `scripts/hil/reflash.sh`:92-127 (indivisible destructive reset and flash). The current `ra8ci` inventory and work-unit JSON are on `ci/orchestrator` at `7ff60fee7`. Unpushed work in other checkouts was not inspected.

Workflow inventory measured on dev for this revision, by parsing `.github/workflows`: 10 files, 1,566 lines, 39 jobs, 39 `runs-on` declarations, one `matrix`, two `needs` edges, ten concurrency groups, about 51 uses: steps, and no `workflow_call` definitions. `firmware.yml` declares 26 of those jobs.

Runner-specific repository evidence on that published dev tip: `infra/fleet.yml` declares ARC on k3s, NAS Docker, Windows/WSL Docker, and a dedicated HIL listener; `infra/ansible/roles/ci_runner/tasks/main.yml` installs the ARC scale set and official runner; `infra/ansible/roles/ci_runner_docker/tasks/deploy.yml` configures per-instance registration and documents `Runner.Listener` / `Runner.Worker`; `infra/ansible/roles/dev_box/tasks/hil_runner_transaction.yml` installs the dedicated listener. GitHub workflows select those runners with `runs-on` labels. These are current-state observations, not authority to alter Brighton's active dev work.

- Build and storage: [Epic 857](#), [Zig build system](#), and [PostgreSQL row locking](#).
- Execution: [Go os/exec](#) and [Go context](#).
- GitHub runner model: [self-hosted runners and autoscaling](#), [scale-set client](#), [runner authentication](#), and [ARC concepts](#).
- Demand signal and gating: [just-in-time runners](#), [workflow jobs REST](#), [CI checks with a GitHub App](#), and [branch protection](#).
- Document format: [Typst PDF documentation](#).